



University of
Sheffield

The Emergent Dynamics of Sequentially Composed Producer-Consumer Systems

Zsigmond Kovacs-Nagy

Supervisor: Charles Grellois

*A report submitted in fulfilment of the requirements
for the degree of*

BEng in Computer Science (with Software Engineering)

in the

School of Computer Science

May 29, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Zsigmond Kovacs-Nagy

Signature:

Zsigmond Kovacs-Nagy

Date: May 29, 2026

Abstract

Sequentially composed producer-consumer systems are present in a wide range of fields, including biology, economics, and computing. To fully understand how these systems behave, a modular approach was taken to investigate the effects of composition of individual producer-consumer systems. As a result, this study provides an empirical analysis evaluating the changes in internal dynamics and system stability caused by system composition. To accomplish this, a simulation framework was developed capable of simulating both open and closed-loop systems with a vast range of configurations. One of the primary discoveries of this study was a significant reduction in system resilience to disturbances after composition. Ultimately, this research is expected to benefit real-world applications such as factory safeguards.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Charles Grellois. His guidance and support throughout this dissertation were invaluable. Additionally, I would like to thank my family and friends for their encouragement.

Contents

1	Introduction	1
1.1	Background	1
1.2	Aims and Objectives	2
1.3	Overview of the Report	2
2	Literature Survey	3
2.1	Atomic Producer-Consumer Systems	3
2.1.1	Architectural Origins and Definition	3
2.1.2	Applications	3
2.2	Sequentially Composed Systems	4
2.2.1	Architecture	4
2.2.2	Applications	4
2.3	Cross-Domain System Classification	5
2.3.1	Rate Classification Using Queueing Theory	5
2.3.2	Control Theory System Classification	6
2.3.3	Higher-Order Systems and The Effect of Delay	7
2.4	Feedback Control Mechanisms	7
2.4.1	Feedback Definition and Implications	7
2.4.2	Types of Feedback Control	8
2.5	Economic Applications and Network Utility Maximization	9
2.6	System Dynamics Classifications	10
2.6.1	Open-Loop Dynamics	10
2.6.2	Closed-Loop Dynamics	10
2.6.3	Closed-Loop Bullwhip Effect	11
2.7	Simulation Methodologies	11
2.7.1	A Simulation-Based Alternative to ODE Limitations	11
2.7.2	Programming Language and Libraries	11
2.7.3	Simulation Computation Paradigms	12
2.8	Summary on Survey	13

3	Approach	14
3.1	Atomic Producer-Consumer Systems	14
3.1.1	Architecture and Rate Formalisation	14
3.1.2	System Classification	15
3.1.3	System Dynamics	16
3.1.4	Throughput	16
3.2	Sequentially Composed Open-Loop Systems	17
3.2.1	Formal Definition	17
3.2.2	System Classification	18
3.2.3	System Dynamics	19
3.2.4	Throughput	19
3.3	Sequentially Composed Closed-Loop Systems	20
3.3.1	Formal definition	20
3.3.2	System Classification	21
3.3.3	System Dynamics	22
3.3.4	Throughput	23
3.4	Simulation Methodology	23
3.4.1	Simulation-Based Evaluation	23
3.4.2	Core Simulation Components	23
3.4.3	Simulation Framework Architecture	24
3.4.4	Simulation Time Progression Mechanism	26
4	Experiments	27
4.1	Experiment Objectives and Evaluation Metrics	27
4.1.1	Theoretical Dynamics Validation	27
4.1.2	Throughput and Queue Occupancy	27
4.1.3	Quantifying Stability	28
4.2	Experiment Process	29
4.3	Open Loop Systems	29
4.3.1	Baseline Configuration	29
4.3.2	Balanced Flow and Aggregate Throughput	30
4.3.3	Bottlenecks and Starvation	30
4.3.4	Backpressure Propagation	30
4.4	Closed-Loop Systems	31
4.4.1	Baseline Configuration	31
4.4.2	Application of Baseline Values and Parameter Exploration	32
4.4.3	Feedback Mechanisms	32
4.4.4	Input vs Output Feedback	33
4.4.5	Sequence Length and Compositional Instability	33
4.4.6	Damping Ratios and Self-Sustained Oscillations	33
4.5	Exogenous Disturbances	33
4.5.1	Baseline Configuration	33

4.5.2	Process Failure	34
4.5.3	Surge Impact	34
5	Results and Discussion	35
5.1	Experimental Correspondence	35
5.2	Open Loop Systems	35
5.2.1	Balanced Flow and Aggregate Throughput	35
5.2.2	Bottlenecks and Starvation	36
5.2.3	Backpressure Propagation	37
5.3	Closed Loop Systems	38
5.3.1	Feedback Mechanisms	39
5.3.2	Input vs Output Feedback	42
5.3.3	Sequence Length and Compositional Instability	43
5.3.4	Damping Ratios and Self-Sustained Oscillations	44
5.4	Exogenous Disturbances	45
5.4.1	Process Failure	45
5.4.2	Surge Impact	46
5.5	Results and Discussion of Limitations	47
6	Conclusions and Future Work	48
6.1	Conclusion	48
6.2	Future Work	48

List of Figures

2.1	First-order queueing system diagram	6
3.1	Atomic producer-consumer system diagram	15
3.2	Sequentially composed open-loop producer-consumer system	17
3.3	Sequentially composed closed-loop producer-consumer system	21
5.1	Open-loop system with parallel processes aggregates the throughput.	36
5.2	Open-loop system containing bottleneck dynamics.	36
5.3	Open-loop system containing starvation dynamics.	37
5.4	Open-loop system containing backpressure propagation dynamics.	38
5.5	Closed-loop atomic system producing stable oscillations.	39
5.6	Closed-loop atomic system stability heat map.	39
5.7	Closed-loop sequential system producing stable oscillations.	40
5.8	Closed-loop sequential system stability heat map.	40
5.9	Closed-loop sequential system with transport delay producing stable oscillations.	41
5.10	Stability heat map of Closed-loop sequential system with transport delay.	41
5.11	Closed-loop sequential system producing partially linear dynamics.	42
5.12	Stability heat map of closed-loop sequential system with input feedback.	42
5.13	Closed-loop five-process system with transport delay producing chaotic dynamics	43
5.14	Stability heat map for a closed-loop five-process system with transport delay.	43
5.15	Closed-loop sequential system producing damping dynamics.	44
5.16	Closed-loop atomic system with process failure	45
5.17	Closed-loop sequential system with process failure	45
5.18	Closed-loop atomic system with a queue occupancy surge	46
5.19	Closed-loop sequential system with a queue occupancy surge	46

List of Tables

4.1	Summary of open-loop baseline parameters.	29
4.2	Summary of closed-loop baseline parameters.	31
4.3	Summary of baseline exogenous disturbance parameters.	34

Chapter 1

Introduction

1.1 Background

Producer-consumer systems are made up of producers, consumers and a shared queue. They follow a pattern where a producer adds produced items to a shared queue, and a consumer consumes items from the queue [1]. This results in a unidirectional flow of information, meaning the consumer is not able to influence the producer or its production rate [1, p. 797]. An example of such a system is the sun, representing a producer, emitting photons into the atmosphere, and a tree consuming the photons for its internal systems.

The pattern for a sequentially composed producer-consumer system is based on linking multiple producer-consumer systems together as subsystems. In particular, the consumer of one subsystem becomes the producer of another, creating one large interlinked chain. This can be demonstrated by extending the previous example so that the tree is the consumer of the photon in one system and also the producer of oxygen in another. By composing subsystems like this, we can explore the dynamics caused by architecture and disturbances on metrics like processing rates, throughput and internal states.

A problem with sequentially composed producer-consumer systems is their susceptibility to bottlenecks and starvations. This is especially apparent within factory lines, which are essentially a physical representation of these systems. Since factories are designed for a steady stream of resources between machines, a small delay in one system can propagate into a cascading stall of the factory. To mitigate against instabilities like this, factories utilise multiple safeguards introduced in Section 2.2.2. While these mitigations do work, understanding the limitations of these composed architectures could reduce waste and improve efficiency for real factories.

Additionally, while there is extensive research on independent producer-consumer systems, the dynamics and optimisation of the composition of these systems are unexplored. However, understanding these dynamics on real-world systems could be risky and costly. As a result, simulating these systems provides a similar alternative without extensive risks or costs. Given that most systems operate interdependently, exploring this domain could uncover dynamics not yet observed.

1.2 Aims and Objectives

The primary aim of this study is to simulate sequentially composed producer-consumer systems in order to empirically analyse their internal dynamics under a variety of configurations and conditions. To achieve this aim, the following objectives must be accomplished:

- Implement a simulation framework with the ability to simulate both open and closed-loop sequentially composed systems.
- Establish a classification framework based on relevant literature to derive the expected dynamics for different system configurations.
- Visualise and analyse the internal dynamics of the simulated systems.

Additionally, the study discusses the potential applications of the research in the domain of factory safeguards and outlines future work based on the Primal-Dual algorithm.

1.3 Overview of the Report

This report contains the following chapters:

- Chapter 2 Literature Survey: Defines the components of the systems, establishes cross-domain correlations to derive expectations, and outlines core implementation concepts.
- Chapter 3 Approach: Outlines the different system types being explored, builds ODE and DDE-based definitions of the systems and uses these definitions in describing how the simulation framework operates.
- Chapter 4 Experiments: Introduces the objective of validating the derived system dynamics expectations, defines the evaluation metrics and the experiments and how they vary from the outlined baselines.
- Chapter 5 Results and Discussion: Presents the results, discusses the comparison between the expectations and observations and outlines successes and areas of improvement.
- Chapter 6 Conclusion and Future Work: Provides an overview of the results of the dissertation and reviews the successful aspects while also providing a roadmap for future work.

Chapter 2

Literature Survey

2.1 Atomic Producer-Consumer Systems

2.1.1 Architectural Origins and Definition

Edsger W. Dijkstra introduced the bounded buffer problem in the 1960s [2]. This problem can be formalised as a producer-consumer system. The components outlined by Dijkstra for such a system are as follows:

- A producer with the ability to add to a queue.
- A consumer with the ability to take from a queue.
- A shared queue between the producer and the consumer.

These three components are connected by the fact that a producer produces items to the shared queue and the consumer consumes items from the shared queue. This creates a unidirectional communication channel. This unidirectionality is reflective of this system's lack of feedback, making it an open-loop system. Additionally, Dijkstra noted that a queue can either be bounded for realism or unbounded for cases where blocking is not desired.

While Dijkstra was the first to formally describe what a producer-consumer system is, his focus was primarily on unbounded queues. Conversely, Downey's work fully explores the implementation and effect of using a bounded queue [3]. Since this study aims to provide realistic solutions, Downey's architecture offers a great basis for the development of the simulation framework.

2.1.2 Applications

Jeffay showed how inefficiency problems like bottlenecks can be visualised and resolved using producer-consumer systems [1]. An example provided by Jeffay referenced a latency issue between the hardware in a head-mounted display and the software computing the visuals. The complex hardware interactions were mapped onto a process graph expressing each component

as either a producer or a consumer. By analysing the data flow, the cause of the bottleneck was identified, and a solution was derived.

While Jeffay focused on the flow restrictions, Szidarovszky emphasised the importance of stability [4]. A study was applied to hypothetical firms producing products based on price for a homogeneous market. The result of extensive analysis showed that if a firm made decisions based on delayed information, it could lose the ability to reach an equilibrium. Additionally, if the delay was significant enough it could lead to instability to the extent of the appearance of oscillations in production rate. These applications show that producer-consumer systems can be used to understand and predict a system's throughput and stability.

2.2 Sequentially Composed Systems

2.2.1 Architecture

A sequentially composed producer-consumer system contains a sequence of interconnected producer-consumer subsystems. This connection is done in a way where the consumer of one system becomes the producer of another. Just like an atomic system defined in Section 2.1.1, this sequential architecture still contains one source process for production and one sink process for consumption. However, it also contains one or more intermediate processes that consume items from the previous queue and produce items to the subsequent queue in the sequence.

Dijkstra's atomic architecture is defined in the domain of Concurrent Computing, where the primary emphasis is on blockage prevention and synchronisation of data. While the dynamics for sequential processes in comparable systems have been explored in other domains, there appears to be a research gap for the composition of discrete producer-consumer systems. More specifically, exploring the effects of composition on the dynamics of the system's internal states.

The most comparable work found on composed architectures and dynamics came from two domains. In queueing theory, a term called a tandem queue is used to describe a sequence of queues where arrival for one queue is passed to the next queue. In control theory, a cascaded system refers to a sequence of connected systems where one output serves as the input of another. These two domains are further discussed in Section 2.3.1 and 2.3.2, respectively.

Additionally, Bruns introduced a term for the intermediate state as a produsage in the domain of social studies [5]. As the name suggests, it describes a collective that is both a producer and a consumer. The application of this term is used to refer to crowdsourcing, where the users consuming a medium are also producing such a medium. An example of such an application was an author reading and writing new Wikipedia articles.

2.2.2 Applications

A real-world application of these composed systems is factory lines. This link can be thought of in such a way that each machine in the factory is a process that is either a producer,

consumer or both, and the conveyor belt between each system can be thought of as the queues in the simulated version. Since an unstable system can cause problems and be less efficient, mitigations were developed to prevent unstable dynamics within the domain of manufacturing operations. Chopra establishes three industrial safeguards within this domain [6]. The idea behind these safeguards, along with their limitations, is as follows:

- **Safety inventory:** A queue of shared resources between subsystems, providing a buffer for times of excess and insufficient supply. However, without resolving the underlying mismatch in resource flow, the static queue can overflow or be depleted.
- **Excess capacity:** This provides excess capacity, allowing for variability in both production and consumption rates, also known as over-provisioning. Given that this safeguard requires the underutilisation of the machinery, there is an inherent cost and inefficiency associated with it.
- **Manual Control:** Relies on engineer supervision to spot and resolve unforeseen conditions outside the scope of automation. The reactive nature of this safeguard can result in delayed responses, leading to severe losses before a resolution.

While the combination of these safeguards provides an effective mitigation for most issues, they are either reactive or resource-intensive. As a result, the understanding of the cause-and-effect of system architecture for such systems has the potential to be useful for real-world applications.

2.3 Cross-Domain System Classification

2.3.1 Rate Classification Using Queueing Theory

The unidirectional architecture provided by Dijkstra means that items flow from the producer to a shared queue and then to the consumer. Both of these processes have production and consumption rates, providing the opportunity for imbalances.

An intuitive domain in which these rates can be modelled through is Queueing Theory. Within Queueing Theory, Kleinrock describes a single-channel queueing system as consisting of a source with an arrival rate of λ and a sink with a service rate of μ . These two components are then connected by a queue with a capacity of K and an occupancy value of $N(t)$ [7]. Given how a single-channel queueing system can define Dijkstra's configuration, this established link can now serve as a source for formal definitions and analytics from queueing theory.

Additionally, Kleinrock described a tandem queue as a system in which queues are linked in a way that departures from one queue move straight onto the next queue [7]. This is the exact process in which items within sequential systems will traverse, and thus provides a robust basis to explore known dynamics within these systems. Since the systems being explored have bounded queues(K), they can be more precisely defined as a bounded tandem queue.

While queueing theory provides parameter definitions, terminologies and a basis for the architecture, its foundations in stochastic and steady-state equations diverge from the intended exploration of transient dynamics. As such, further dynamics exploration will rely on control theory.

2.3.2 Control Theory System Classification

While Dijkstra described the system parameters and Kleinrock formally defined the flow parameters, they do not cover the dynamics resulting from this configuration. Within Control Theory, Åström and Murray describe the dynamics of such systems.

Åström and Murray introduced two terms to describe and classify systems called state variable (x) and state vector ($\mathbf{x}$) [8]. For a system, the state variables are the individual stores of momentum, mass or energy, depending on the system. Conversely, the state vector for a system is the collection of all the state variables. When identifying a system, the size of the state vectors is often used, and thus, first-order systems refer to systems with one state variable and so on.

Åström and Murray define a first-order system as one where the state vector is captured by a single variable [8]. These first-order systems can be modelled by the following ordinary differential equation (ODE):

$$\frac{dx}{dt} = \lambda - \mu \quad (2.1)$$

Where x is the state variable representing the queue occupancy, λ is the arrival rate, and μ is the service rate. Since a system's time-dependent queue occupancy is represented as $N(t)$ in queueing theory and $x(t)$ in control theory, these two parameters from different domains are mathematically equivalent. Expanding on this equation (2.1), the state variable can be visualised as shown in Figure 2.1.

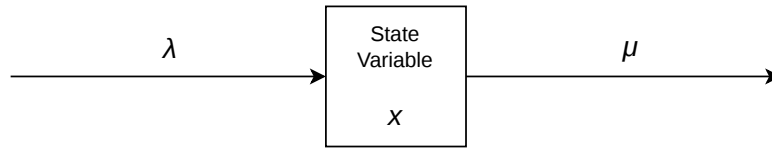


Figure 2.1: First-order queueing system diagram

This definition of a first-order system aligns with Dijkstra's configuration, as the system's momentum is defined by a single variable, which is the queue occupancy. Also using Åström and Murray's definitions, Dijkstra's configuration is an open-loop system as the production rate (λ) and the consumption rate (μ) are independent. For closed-loop systems, the arrival rate and the service rate are linked in such a way to allow feedback.

2.3.3 Higher-Order Systems and The Effect of Delay

A higher-order system is one where the size of the state vector is two or more. Mathematically, this is equivalent to a system described by a differential equation containing a second or higher-order derivative. Since each queue occupancy is a state variable, a composed system is classified as a higher-order system. More precisely, control theory provides a specific term to express systems containing multiple connected state variables called cascaded systems [8]. Additionally, a system's phase lag is defined as the time difference between the system's actual state and the delayed state that the feedback controller receives to use for its response. This means that if a system contains phase lag, it cannot make instantaneous decisions based on the latest state information. As a result, systems with excessive phase lag overcorrect and become unstable.

This behaviour is formalised by Stermann, who states that to achieve complex dynamics like oscillations, at least a second-order system is required [9]. Building on this, Åström and Murray establish that a phase lag of 180 degrees is required for oscillations to occur [8]. This phase lag can either come from the use of integral control in the feedback or from transport delay (δ) in the feedback calculations.

While both integral control and transport delay introduce phase lag, the accumulated error in integral control adds an additional single state variable to the system, while the transport delay represents an infinite-dimensional state variable. In addition, a system using integral control can be modelled using an ordinary differential equation (ODE), whereas a system containing transport delay is modelled as a delay differential equation (DDE) [10]. This identification is crucial, as going from a single state variable to infinity would make it harder to know if the change in dynamics was a result of the architecture or the underlying mathematics. By tracking the progression from first-order to higher-order and then infinite-dimensional systems, the influence of both the architectural and mathematical differences can be studied separately.

2.4 Feedback Control Mechanisms

2.4.1 Feedback Definition and Implications

To formally define feedback in the domain of control theory, the associated terminology has to be established. Åström and Murray defined the reference signal (r) as the desired value of a state variable and the system output (y) as the state variable that the target is applied to. The objective of feedback is to minimise error (e), which corresponds to the difference between the reference signal and the system output [8]. This can be expressed as $e = r - y$. Knowing this terminology, the three types of feedback introduced by Åström and Murray can now be formally defined:

- **Feedforward:** A predictive architecture in which the disturbance is measured before entering the system. These measurements are then used to predictively correct the

effect of the disturbance to track the reference signal. This is especially useful for responses to command signals.

- Negative feedback: A reactive system that uses the error as a corrective effect to return the system to the reference signal. This is the primary mechanism for systems like supply chains to regulate their internal states.
- Positive feedback: A mechanism that amplifies a disturbance in such a way that an increase will result in a further increase. This mechanism is extensively used in biological systems.

The objective of most real producer-consumer systems is to provide high throughput at an efficient rate. To achieve this, negative feedback is an effective solution as it can be used to discourage starvation and bottlenecks using the reference signal. Additionally, the target value can be set in such a way that disturbances are absorbed into the state variable and mitigated by the feedback afterwards.

There are some advantages and drawbacks to introducing feedback to a system. The primary advantage is the adaptability the system gains to react to disturbances. On the other hand, Åström and Murray note that feedback does have a chance to introduce instability, noise and coupling of components [8]. Typically, the advantages outweigh the drawbacks, and as a result, a system with feedback becomes more robust and stable. Additionally, this coupling introduced by feedback can result in the dissolution of the previously established unidirectional nature of information in a producer-consumer system. This means that the system architecture can become multi-directional, allowing for coupled rate adjustments.

While in most instances stability is the desired outcome, for the purposes of this study, instability will be the focal point. Finding the instability that occurs as a consequence of architecture or configuration parameters can offer insight into what to avoid and, in the case of instability, what to expect.

2.4.2 Types of Feedback Control

To apply negative feedback, a controller and an actuator are required. The controller contains the control law and is used to apply this law to the error (e) to obtain the control signal (u). The control signal is designed to reduce error and is applied to the system using the actuator. There are many different control laws for different purposes. For this study, the following four laws defined by Åström and Murray [8] will be introduced here:

- On-Off Control: This is a simple feedback mechanism that can be expressed as:

$$u = \begin{cases} u_{max} & \text{if } e > 0 \\ u_{min} & \text{if } e < 0 \end{cases} \quad (2.2)$$

As showcased by Equation 2.2, on-off control always applies the maximum correction possible. While this does keep the system near the reference signal, it usually results in small oscillations.

- P Control: This feedback mechanism adjusts the control signal proportionally to the error. The law can be defined as:

$$u = k_p e \quad \text{for } e_{min} \leq e \leq e_{max} \quad (2.3)$$

Where k_p is the proportional gain, which reflects a reaction sensitivity of the feedback. For real-world systems, proportional bands can be used to bind the values of u between e_{min} and e_{max} , which reflect the system's limitations. This bounding can also be expressed as saturation limits, bounding the control signal to not exceed u_{max} or u_{min} . These proportional bounds and saturation limits can be and should be applied to the other PID-based control types to limit runaway reactions. P control is a significant improvement from on-off control and provides smooth adjustments to the reference signal. However, it can suffer from steady-state error, which means the system can stabilise at an offset from the reference signal.

- PI Control: Builds on P control by using an integral term, allowing control decisions based on the historical accumulation of error. This control law is expressed as:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau \quad (2.4)$$

Where k_i is the integral gain and $\int_0^t e(\tau) d\tau$ is the integral of the error over time. PI control uses the sum of P control with the new integral and gain value. The integral of the error over time is equivalent to a state variable and, as such, can store accumulated memory of all past deviations. This allows for the correction of the steady-state error P control suffers from.

- PID Control: This is the industry standard feedback used in most circumstances. It incorporates three control laws and can be expressed as:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt} \quad (2.5)$$

Where k_p , k_i and k_d are the gain values for the three control laws. As shown in Equation 2.5, PID control combines the two previous control equations with derivative control. This allows the control to influence the control signal based on the past depicted by the integral, the present depicted by the proportional component and the future depicted by the derivative. PID control eliminates the steady-state error from P control and provides an anticipatory effect not found in PI control.

2.5 Economic Applications and Network Utility Maximization

A tangential economics approach to the bounded buffer problem is the framework of Network Utility Maximisation (NUM) by Kelly [11]. The goal of this framework is to maximise the

value based on the utility, price, production rates and queue occupancy. Using the NUM objective function to achieve this maximum utility creates a shadow price used to arrive at a stable global optimum.

Expanding on Kelly's work, Low and Lapsley introduced a Primal-Dual algorithm which is capable of applying the NUM optimisation framework to individual processes [12]. As a result, this Primal-Dual algorithm can be an alternative to PID-based feedback. Given time limitations and a general divergence from the scope of the study, the implementation and use of this algorithm were labelled as future work.

2.6 System Dynamics Classifications

2.6.1 Open-Loop Dynamics

A system that does not use feedback is considered an open-loop system. Åström and Murray state that the mechanisms of these systems lack the capability to react to changes in their state [8]. As a result, the processing rates within these systems are constant, with the exception of rate limiting caused by the bounds of a queue. This rate limiting is either called starvation, when the lack of input items is throttling the processing rate, or a bottleneck when the output queue is full, blocking production. This means that the state variables follow linear trajectories.

2.6.2 Closed-Loop Dynamics

Closed-loop systems are influenced by feedback and so can correct for disturbances by reacting to changes in their internal states. Åström and Murray defined a useful metric called damping ratio (ζ), which can be used to categorise a system's dynamics [8]. The damping ratio reflects a system's resistance to change and can be expressed as:

$$\zeta = \frac{c}{2\sqrt{km}} \quad (2.6)$$

Where c represents the damping coefficient, which acts as a shock absorber, k represents the reaction sensitivity, and m represents the system's mass.

Calculating this ratio for closed-loop systems allows the separation of the systems into two different bands with different properties and dynamics [8]. These bands are:

- Overdamped and critically damped: Overdamping occurs when the damping ratio is greater than or equal to one ($\zeta \geq 1$), and critically damped is when the ratio is exactly 1 ($\zeta = 1$). These are known as asymptotically stable and so smoothly approach the target reference, never exceeding it.
- Underdamped: Underdamping happens when the damping ratio is below 1 ($0 < \zeta < 1$). This results in overshooting dynamics as the system lacks the required resistance to counteract the system's momentum in time. Without phase lag, also known as either

physical integral control or latency, these systems act as a damped oscillator. However, if there is a phase lag in the system, sustained oscillations are possible.

2.6.3 Closed-Loop Bullwhip Effect

The Bullwhip Effect is a supply chain phenomenon caused by continuous increases in the variability of supply and demand that travels up the sequence. To induce the formation of the Bullwhip Effect, Lee et al. identify several necessary requirements. These include a sequential supply chain, a significant lead time between ordering and receiving goods, multiple individual feedback mechanisms and varying demand [13]. The produced dynamic is based on the amplification of variance, where the related nature of the sellers and buyers results in the compounding of error.

Extrapolating these requirements into a producer-consumer domain means that for a system to have the chance to produce the Bullwhip Effect, it requires the following. A sequence of interconnected producer-consumer systems with individual feedback mechanisms and the existence of transport delay (δ) between processes. This type of system can be expressed as a DDE, introduced in Section 2.3.3. Given the scope of this study, these requirements should be achievable and show how individual local feedback can induce global dynamics.

2.7 Simulation Methodologies

2.7.1 A Simulation-Based Alternative to ODE Limitations

Since a connection was established between composed systems and ODEs in Section 2.3.2, it would seem reasonable to conduct the experiments in this study by solving these equations. However, as these composed systems scale, the computations required to solve these ODEs become highly complex and expensive. As noted by Segovia, a solution to mitigate this increased complexity is to use flow-based network models, like Petri nets, to simulate the underlying dynamics without the need for numerical integration [14]. Additionally, Segovia established a correspondence between ODEs and flow networks, while also stating that Petri nets can offer an efficient alternative to ODEs for evaluating evolutionary processes. Having established the possibility of implementing ODEs as flow-based networks, this study will use a flow-based discrete-event framework.

2.7.2 Programming Language and Libraries

Since this project's main aim is to create an empirical study of a programmed simulation, choosing an appropriate programming language is essential. To ensure an appropriate choice, Nanz and Furia's [15] comparative study on different programming languages was leveraged.

The study found that while code from languages like Perl and Python was roughly twice as concise as code from languages like C/C++ and Java, C still outperformed the other languages in terms of performance. While performance is key, the study noted that the performance of

Python is sufficient for most use cases. Using this study as a guide, Python was chosen for its fast development and reasonable performance.

Choosing Python provides the benefit of its vast and well-supported libraries. For this project, the prominent libraries and built-in modules used are:

- NumPy: Used for data handling and operations.
- Matplotlib: Provides a visual representation of large datasets like the simulation results.
- Optuna: Is an open-sourced framework that offers a Bayesian optimisation tool. It can be used to tune the provided parameters to maximise the results towards the outlined objective [16].
- Data Classes: Is used to provide formally expressed data structures essential for the processing of all the data types necessary for simulations [17].

2.7.3 Simulation Computation Paradigms

Python-based simulations either use real-time computations or simulated time computations. To perform real-time computations at a large scale, either multithreading or multiprocessing can be used. While for simulated time computations, time steps can either be constant or event-based.

Real-time computations rely on the physical passage of time. Since this study aims to explore the dynamics of the system's internal states, having a deterministic time is essential, especially for the implementation of feedback. Given this, a delay has to be used to reflect the processing time of each item. To implement these types of systems, either multiprocessing or multithreading can be used to eliminate bottlenecks caused by multiple processes running at the same time. Brownlee defined a way to implement producer-consumer systems using both multiprocessing and multithreading [18] [19]. These implementations provide a basic understanding of the potential implementation of such systems and use `time.sleep()` to represent processing times. Additionally, Brownlee's examples correspond to how Downey described the development of bounded queues [3].

Simulated time implementations require an internal time controller, which is responsible for the advancement of time and running of the processes at their allocated times. To progress this time, it can either be done using incremental time advances or an event-based system that increments time to the soonest action. Banks establishes that an event-based solution is considerably better as it eliminates redundant increments and stops the bucketing of actions per step [20].

While event-driven simulated time is the most appropriate solution for this dissertation, the decisions leading up to its implementation are further discussed in the Approach chapter, Section 3.4.4.

2.8 Summary on Survey

This chapter encompasses the necessary theoretical and architectural literature needed to establish the notion of sequentially composed producer-consumer systems. A foundational approach was taken in which the original producer-consumer definition was extended with mathematical formulas and correlations. These came from cross-domain mappings from both control theory and queueing theory. Having defined these foundations, the addition of localised feedback and the extrapolation of expected system dynamics were possible. Furthermore, computing paradigms and implementation tools were discussed.

Chapter 3

Approach

3.1 Atomic Producer-Consumer Systems

3.1.1 Architecture and Rate Formalisation

Within this dissertation, multiple sequentially compatible producer-consumer systems will be composed to explore their dynamics. Hence, having a solid basis for an individual producer-consumer system is essential to achieve sound results. To do this, Dijkstra's configuration from Section 2.1.1 will be used to model such systems[2].

To model the flow through an individual system, we can correlate Dijkstra's configuration onto a single-channel queueing system described within Section 2.3.1. Using Kleinrock's definition of a single-channel queueing system, we get the following correlation:

- Production rate = Arrival rate (λ)
- Consumption rate = Service rate (μ)
- Queue capacity = Storage capacity (K) (infinite or bounded)
- Queue occupancy = State variable (x)

While in queueing theory, the time-dependent queue occupancy is denoted as $N(t)$, for the purposes of this dissertation, $x(t)$ will be used. As discussed in Section 2.3.2, these values are interchangeable, and since future work will rely on control theory expressions, choosing to standardise the notation seems appropriate. This atomic architecture is illustrated in Figure 3.1, which depicts the unidirectional nature of these components and how they interact. The term process will be used as an abstraction for both producers and consumers. This will assist in describing later figures, which contain intermediate processes that are responsible for both production and consumption. As such, in the figure, Process 1 acts as a producer with a processing rate of λ and Process 2 acts as a consumer with a processing rate of μ . These two processes are then linked by a queue with a capacity of K and an occupancy of x .

While systems within queueing theory primarily use stochastic values for arrival and service rates, for the purposes of this dissertation, deterministic rates will be used. This

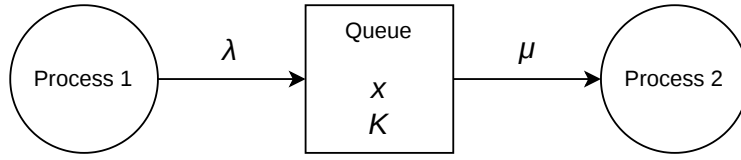


Figure 3.1: Atomic producer-consumer system diagram

decision removes random variables, meaning that the observed dynamics will be reproducible. Even though Dijkstra called the store between the producer and consumer a buffer, for the application of this dissertation, the term queue is more appropriate.

3.1.2 System Classification

As stated in Section 2.3.2, Dijkstra configuration can be correlated to Åström and Murray's classifications to give an insight into the possible dynamics. This correlation will not only be helpful for this configuration, but also for the subsequent configurations based on the configuration described here.

Based on Åström and Murray's definitions, we can model Dijkstra's configuration as a first-order open-loop system. Since it's a first-order system, it has one state variable (x), which is its queue occupancy. This correlation does come with two concerns that need to be addressed. First, to simulate these dynamics, I will use discrete items, but Åström and Murray's equation uses continuous time. Using discrete changes instead of continuous changes means that the trajectory of the dynamics modelled by the equation is preserved. The only concern is if the discrete changes are too coarse. A mitigation for coarse dynamics is ensuring that the system is using enough items, as with more items, the rate of change at each step will be smoother. The second concern is that the equation assumes the queue is unbounded, whereas for the simulation, it is actually bounded. The choice to make the queue bounded was for realism and applicability to real systems, reflective of Dijkstra's view on the topic. Having the queue bounded for an atomic producer-consumer system results in:

- Linear Flow, if the queue is neither full nor empty. Given this state, the dynamics of the equation are not impeded by the bounds of the queue.
- Starvation, if $\lambda < \mu$ and the queue is empty. This is because the consumer is bottlenecked by a lack of production rate. Also, for an atomic producer-consumer system, if $\lambda < \mu$, the queue occupancy will always tend towards starvation.
- Blocking, if $\lambda > \mu$ and the queue is full. This is because the producer is bottlenecked by the lack of consumption rate. Also, for an atomic producer-consumer system, if $\lambda > \mu$ the queue occupancy will always tend towards saturation.

After addressing these concerns, the dynamics of the configuration's state variable (x) are defined in the Ordinary Differential Equation (3.1). Given that the state variable corresponds

to the queue occupancy, this equation expresses the change in queue occupancy depending on its internal state.

$$\frac{dx}{dt} = \lambda - \mu \quad (3.1)$$

Where:

$$\frac{dx}{dt} = \begin{cases} 0 & \text{if } x = K \text{ and } \lambda > \mu, \\ 0 & \text{if } x = 0 \text{ and } \lambda < \mu, \\ \lambda - \mu & 0 < x < K \end{cases}$$

To provide a deterministic starting point, the initial state for this and subsequent systems will be defined as $x(0) = 0$. Having expressed Dijkstra's configuration within the scope of this study provides a substantial foundation for further work.

3.1.3 System Dynamics

Now knowing the precise equation responsible for the queue occupancy within Dijkstra's configuration, we can analyse it to understand the underlying dynamics available within this configuration. As outlined by the first-order equation, there are three apparent dynamics that can occur:

- If $\lambda < \mu$, the queue occupancy will linearly decrease until the queue becomes empty ($x \leq 0$). After which, the queue will remain in a depleted state.
- If $\lambda = \mu$ the queue occupancy stays in equilibrium.
- If $\lambda > \mu$, the queue occupancy will linearly increase until the queue occupancy (x) saturates to the point of the queue capacity (K), which can be expressed as $x = K$. After which, the queue will remain in a saturated state.

Conversely, the minimum threshold required for more complex dynamics is defined in queueing theory (Section 2.3.1), was not met for this architecture. To get complex dynamics like oscillations, overshoots and the Bullwhip Effect, the system needs to be a closed-loop, have a phase lag and have at least two state variables.

3.1.4 Throughput

Throughput is the flow that can pass through the system, from start to end. As a result, the minimum rate is the limiting factor. This can be showcased as:

$$T = \min(\lambda, \mu) \quad (3.2)$$

This means that regardless of the internal state, the minimum throughput will always be achieved. This will be reflected in the experiment stage, in which we will see how throughput for such a system will always converge to the local system minimum given enough time.

3.2 Sequentially Composed Open-Loop Systems

3.2.1 Formal Definition

Building on the concept of an atomic producer-consumer system, the composition of such systems can be explored. Instead of treating a sequence of atomic systems as individuals, they can be treated as a single chain of atomic subsystems. For a sequence like this, the subsystems link in such a way that the consumer of one system becomes the producer of another. An intuitive example of a composed system is a factory line, where each machine can either be responsible for the production of an item, the consumption of an item or both, depending on its position in the sequence. For such a sequence, the first machine is only responsible for production and thus introducing items into the wider system, and the last machine is solely responsible for consuming and thus resulting in the finished product from that factory line. All of the intermediary machines are therefore responsible for both the consumption of items in one queue and the production of items in another queue.

To illustrate this sequential architecture, Figure 3.2 demonstrates the composed nature of these subsystems. The figure contains two atomic systems, which in the wider system correspond to two subsystems. These subsystems form a sequence of three processes, where Process 1 is a producer, Process 2 is an intermediate process responsible for the transfer of items and Process 3 is a consumer. As denoted in the figure, the initial producer and the final consumer have independent processing rates, but this is not the case with Process 2, as just like in a factory setting, an item needs to be consumed for another to be produced. This means that the consumption rate of the first subsystem (μ_1) is equivalent to the production rate (λ_2) of the second subsystem, which can be expressed as $\mu_1 = \lambda_2$. Lastly, each queue has its own independent capacity (K_i) and occupancy (x_i).

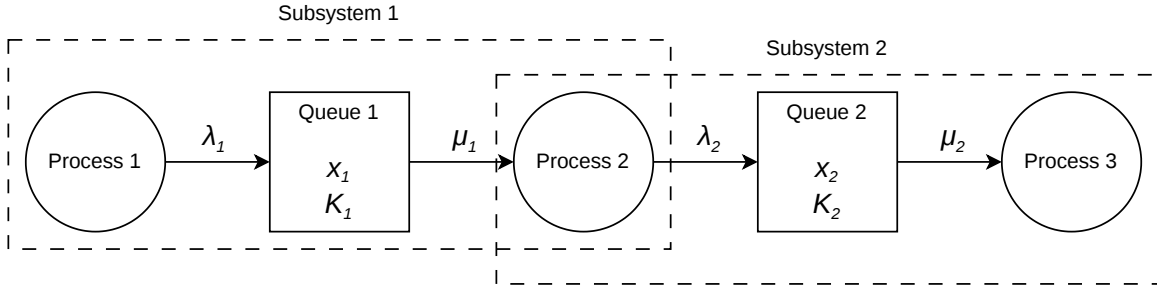


Figure 3.2: Sequentially composed open-loop producer-consumer system

Given this design, for a sequence with n queues and m processes, an intermediate process consuming from queue i at a consumption rate (μ) also produces to queue $i + 1$ at the same production rate (λ). This can formally be described as:

$$\mu_i = \lambda_{i+1} \quad \text{for } i \in \{1, 2, \dots, n-1\} \quad (3.3)$$

The formula ends at $n - 1$ as the final process only consumes, and so no production rate is needed for the next subsystem. Importantly, this equation represents an unbound flow, and thus either starvation in the input queue or a bottleneck in the output queue can become the limiting factor. While these constraints have been discussed in more depth in Section 3.1.2, this equation provides insight into the dynamics between sequential queues.

Lastly, this configuration still retains its unidirectional flow and open-loop nature inherited from its atomic subsystems. Due to the bounded queues (K) within each subsystem, full queues will block upstream processes, providing realistic back-pressure dynamics. However, given the lack of feedback, this configuration is an open-loop system.

3.2.2 System Classification

A definition for sequentially composed systems has been established in both queueing theory in Section 2.3.1, and control theory in Section 2.3.2. Within queueing theory, they are called bounded tandem queues, and in control theory, they are called higher-order systems. Having established these definitions, they offer both a framework to model their behaviours and thus an insight into the available dynamics they can produce.

The continued bounded nature of the queues in the system means that the established constraints in Section 3.1.2 of starvation and blocking remain. Additionally, the increased sequence length now means that in the event of starvation and blocking, both upstream and downstream dynamics can be observed.

To calculate the queue occupancies for a sequential system with n queues, the atomic Equation (3.1) can be integrated with the logic from Equation 3.3. Since the first queue ($i = 1$) has no preceding consumers, its arrival rate depends solely on the production rate of the first process (λ_1). Resulting from the integration, the subsequent queues ($i > 1$) can now be expressed just by the consumption rates within the system.

$$\frac{dx_1}{dt} = \lambda_1 - \mu_1 \quad (3.4)$$

$$\frac{dx_i}{dt} = \mu_{i-1} - \mu_i \quad \text{for } i \in \{2, 3, \dots, n\} \quad (3.5)$$

In these expressions, the interconnected nature of these systems can be formally expressed. Explicitly, it states that for an intermediate queue i , its queue occupancy (x_i) is not only dependent on the consumption rate (μ_i) within that subsystem, but also the consumption rate from the previous subsystem (μ_{i-1}).

Given the architecture of these composed systems, each subsystem contains an individual queue corresponding to a state variable. As a result, if there are n subsystems, then the system classification for such a system will be an n -th order open-loop system. This can be expressed as:

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix} \quad (3.6)$$

Where $\mathbf{x}$ is the state vector at time t , which is responsible for containing all the states of the state variables (x) at time t . Additionally, each state variable starts empty, which can be expressed as $\mathbf{x}(0) = 0$. Since these are open-loop systems, it means that the state variables are not affected by feedback and thus experience linear changes within the bounds of each queue's state.

3.2.3 System Dynamics

The composed architecture of this open-loop system faces the same limitations introduced in Section 3.1.3 and thus experiences linear flow, bottlenecks and starvation. While the lack of feedback means no complex dynamics are possible for these configurations, there is a significant impact of composition.

The largest difference from the dynamics of atomic systems is the ability for the dynamics to propagate. This means that, for example, a bottleneck at the end of the system does not immediately reflect at the start of the system. In this scenario, a wave of saturations propagates upstream. It is noteworthy that since each simulation begins with an empty state vector ($\mathbf{x} = 0$), the inverse of this example does not result in propagation. This is a consequence of the limiting factor of the minimum processing rate established in Equation 3.2. As a result, the inverse example would result in constant processing rates bounded by the bottleneck rate. Additionally, the increased length of these systems can now express both the upstream and downstream consequences of dynamics like starvation and bottlenecks.

While the propagation is a new dynamic, it is not a sustained event. Consequently, once the system reaches a stable state, it will remain there. The internal dynamics of each state variable follow Equation 3.1 with the caveat that the processing rates can now be throttled by the state of its immediate neighbours.

3.2.4 Throughput

The global throughput (T) of a system is defined by its slowest processing rate. This can be expressed for a system with n processes as:

$$T = \min(\lambda_1, \mu_1, \mu_2, \dots, \mu_n) \quad (3.7)$$

This equation conveys that while internal dynamics may vary, the throughput for such systems will always be equivalent to T . While the internal subsystems might experience higher throughput during the transient state of the system, each subsystem will eventually converge to T . The sequence length can affect the speed of convergence, but not its end state.

3.3 Sequentially Composed Closed-Loop Systems

3.3.1 Formal definition

Transitioning to the exploration of closed-loop composed systems introduces the idea of feedback. In Section 2.4.1, the decision to use negative feedback was discussed, and in Section 2.4.2, multiple negative feedback controls were introduced. Out of these control types, PI control was chosen as the basis of the experiments in this study. Additionally, PI control will be used to explore the effects of transport delay (δ) introduced in Section 2.3.3. The decision to use PI control over the more standard PID control was driven by the following considerations:

- **Stability:** In terms of stability, PID is a definite improvement. While PI control does reach stability using a smooth and effective gradient, it does lack the predictive dampening used by PID.
- **Complexity and multiple dimensions:** To use PID, three distinct gain values and two state variables are required. In comparison, PI control only needs two gain values and one state variable. These extra parameters required by PID control not only increase complexity but also contribute to an increase in state variables.
- **Dynamics:** Given the improved stability PID provides, there is a reasonable chance that dynamics occurring from the system architecture and its configuration could be mitigated by such an effective feedback system. Given that the aim of this dissertation is the exploration of dynamics induced by such systems, this improved stability is undesirable for the purposes of this study. While PI control may result in more dynamics, it is by no means an inferior type of control and is rather a more transparent system that allows for the occurrence of dynamics if the architecture induces it.

To prioritise clearer correlations with fewer dynamics-inhibiting factors, PI control was chosen as the baseline for the study's experiments. As discussed in Section 2.3.3, either integral control or transport delay is required to induce phase lag, which is responsible for complex dynamics. Given this, for closed-loop experiments, PI control and PI control with transport delay will be explored to understand the impact of different feedback configurations.

Extending upon the open-loop architecture definition in Section 3.2.1, the closed-loop counterpart introduces new relationships between processes and queue occupancies. Since closed-loop architectures rely on the use of feedback, processing rates in these systems change based on factors like the queue occupancy and the feedback parameters. To illustrate these changes, Figure 3.3 shows a sequentially composed system with the addition of feedback control.

The figure depicts the previously defined unidirectional flow of items using solid arrows, while dashed arrows are used to show the new flow of information. The figure's configuration applies the reference signal (r) to the output queue and is configured to allow for PI control with transport delay and all of its subsets, depending on the parameters applied. The figure

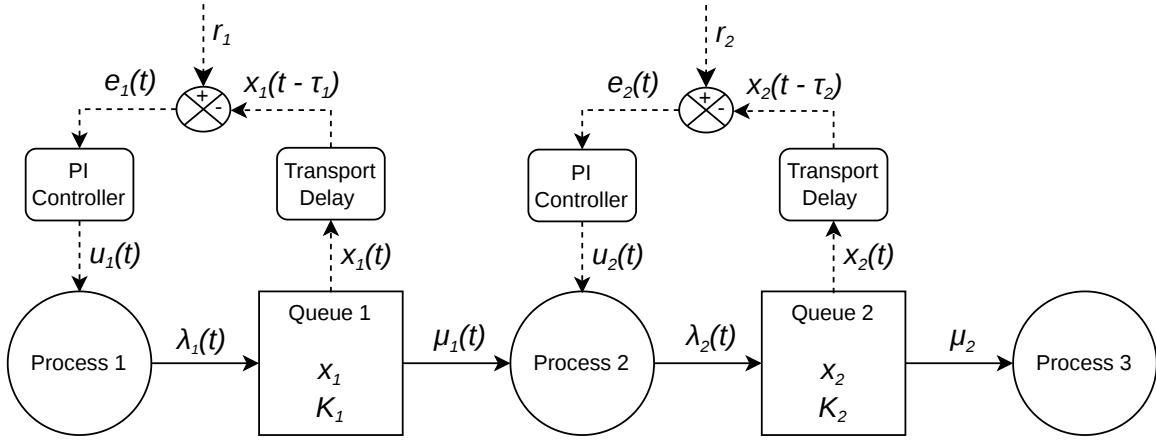


Figure 3.3: Sequentially composed closed-loop producer-consumer system

shows each queue's state variable as x_n , where n is the index of the queue. The values of these state variables are time-dependent, and as such, the value of the variable is expressed as $x_n(t)$. Given this, processes that utilise feedback now have time-dependent processing rates ($\lambda_n(t)$ or $\mu_n(t)$). The update mechanism for these feedback-driven processing rates is based on the state variable's error ($e(t)$), which is calculated as the difference between the reference signal and the queue occupancy that the feedback controller receives ($x(t - \delta)$). The delay between the current queue occupancy and the value received by the controller is represented by δ , which denotes the transport delay. Expanding on this figure, for such systems, the normalised error which drives the feedback mechanism can be formally defined as:

$$e(t) = \frac{r - x(t - \delta)}{r} \quad (3.8)$$

The objective of Equation 3.8 is to calculate the error, which is then used by the PI controller to find the control signal ($u(t)$) responsible for the corrective force needed to reach the reference signal. The error is normalised to ensure consistent values, regardless of scale.

3.3.2 System Classification

In Section 2.3.3, it was introduced that the number of state variables in a system corresponds to the system's order. As such, an open-loop sequentially composed system with two queues is a second-order system. While in previous sections, the addition of more queues was the only way to increase a system's order, the introduction of closed-loop parameters offers two more alternatives for this study. These are the accumulated errors from the integral term in PI control and the effect of transport delay on the feedback controller. Both of these alternatives are types of phase lag. While PID does introduce more state variables, it is outside the scope of this study and so will not be considered.

While phase lag provides a way to increase the number of state variables, it is also

essential for oscillator dynamics, as discussed in Section 2.3.3. The minimum threshold for such dynamics is defined by Sterman, who states that a system of at least a second-order system is needed for oscillations [9], and Åström and Murray establish that a phase lag of at least 180 degrees is required for sustained oscillations [8]. In Section 3.3.1, it was established that closed-loop experiments will contain at least one type of phase lag, satisfying Åström and Murray’s threshold. Furthermore, both types of phase lag result in second-order or higher systems, satisfying Sterman’s threshold.

To derive the state variable introduced by the error integral in PI control, the error Equation 3.8 needs to be extended. Up until this point, the state variables referring to the queue occupancies have represented the system state, denoted by x . However, the error integral is not a store of the system state, but instead the feedback state. Consequently, a different state variable of z is allocated to this store. Since state variables, like the queue occupancy, store the magnitude of the difference between processing rates, this can be applied to the accumulated error (z). This rate of change of this state can be expressed as:

$$\frac{dz}{dt} = e(t) \quad (3.9)$$

This means that for an atomic system using PI control, the state vector can be expressed as:

$$\mathbf{x}(t) = \begin{bmatrix} x(t) \\ z(t) \end{bmatrix} \quad (3.10)$$

As discussed in Section 2.3.3, the use of transport delay results in an infinite-dimensional state variable. This is due to the continuous nature of history, which is different from the discrete integral error variable. The use of transport delay is also a component for the Bullwhip Effect discussed in Section 2.6.3. To achieve this dynamic, the required architecture includes the combination of feedback, transport delay and sequentially composed systems. Consequently, the integration of all the components defined in this chapter is required to successfully achieve this dynamic. While creating Bullwhip Effects was a desired outcome for this study, it will be reserved for future work due to time limitations.

Even though the effect of transport delay will be explored, the complexity needed to derive an equation for such systems was deemed excessive and outside the scope of this study.

3.3.3 System Dynamics

The catalyst for the dynamics in closed-loop systems is their feedback mechanism. As discussed in Section 3.3.1, this study will use a feedback controller based on PI control, with the optional addition of transport delay (δ). The application of this feedback controller is crucial as it will determine how the processing rates change based on the error values. So for the implementation of the simulation controller, the PI control equation introduced in Section 2.4.2 was used, with a small adaptation. Since these equations are in continuous time and the simulation is in discrete time, expressing the integral of the error in the equation using the

new state variable z introduced in Section 3.3.2 bridges this gap. This can be expressed as:

$$u(t) = k_p e(t) + k_i z(t) \quad (3.11)$$

To apply this feedback to the processing rates, a linear rate-scaling method was used. This is done by multiplying the base rate of the process by the control signal (u). If the base processing rate of a system is defined as r_0 , then this application formula can be expressed as:

$$r(t) = r_0(1 + u(t)) \quad (3.12)$$

Additionally, to prevent runaway reactions, saturation bounds introduced in Section 2.4.2 are applied to the control signal (u). A realistic bound of $u_{max} = 1$ and $u_{min} = -1$ was applied. These values seemed appropriate as at a control signal of $u = 1$, the processing rate is 200% and at a signal of $u = -1$, the processing rate is 0%.

3.3.4 Throughput

In this closed-loop architecture, processing rates change and might not ever settle. As a result, instead of finding the smallest processing rate as defined for the open-loop systems, the entire system's processing rate needs to be considered. While stable systems can converge onto the reference signal, making the system throughput apparent, for oscillatory systems, finding the system throughput would be mathematically complex and outside the scope of this study.

3.4 Simulation Methodology

3.4.1 Simulation-Based Evaluation

While the system architectures introduced in this chapter have been established via Ordinary Differential Equations (ODEs), solving these equations becomes computationally expensive. This was discussed within Section 2.7.1, where flow-based networks were introduced as an alternative to ODEs. The apparent benefits of these flow networks are their emphasis on rules-based simulations, without the need for complex mathematical computations. This was expressed by Segovia's reference to Petri nets, which are a subset of flow networks, as a playing board on which the evolution of phenomena can be simulated [14].

Considering these two options for the implementation of this study's experiments, flow networks were chosen for their computational efficiency and easier correlation to the system architectures.

3.4.2 Core Simulation Components

The supporting literature for the decisions made for the implementation of this simulation framework can be found in Section 2.7. Based on that information, the implementation is written in Python and uses libraries including NumPy for log data handling, Matplotlib for

plotting simulation results and Optuna for automated parameter tuning towards oscillatory behaviours [16]. Data classes are used to encapsulate data and to define a strict separation of concerns [17]. The three main components driving the simulations are as follows:

- The producer, responsible for producing the allocated items to a shared queue with the consumer. The rate at which these happen depends on a constant production time for each item and whether feedback is enabled. Within the context of this simulator, each producer has an output queue to which it produces items.
- The consumer, responsible for the consumption of items from the allocated queue. A consumer can either be a sole consumer or both a consumer and a producer. This depends on whether, for the given system that is being simulated, the consumer in question is also expected to use the consumed item to produce something else within the system as a whole. This allows the sequential composition of individual producer-consumer subsystems, and for the sequence to end at the last consumer with no output queue. Within the context of this simulator, each consumer has an input queue from which it consumes and has the option to have an output queue if the consumer acts as a producer to the output queue.
- The simulation runner, responsible for running the simulation based on the architecture defined within the system configuration data class. The simulation is event-driven and thus uses minimal computational overhead and provides the clearest results.

3.4.3 Simulation Framework Architecture

The framework is split into three files, where each of them has a specific responsibility, ensuring clear separation of concerns. The first one contains all of the simulation logic, feedback control, data logging and visualisation tools, called `sim_runtime.py`. The second file, called `sim_dataclasses.py`, defines the data classes used as the basis of the system architecture and as the store for the live simulation data. The last file `sim_scenarios.py` stores the simulation experiments in individual functions which can be called by the runtime, returning both the simulation configuration and the stability configuration. The simulation configuration contains the parameters needed to define the architecture of the simulation, whereas the stability configuration contains the size and axis of the stability heat map used for that simulation.

The data classes inside `sim_dataclasses.py` can be grouped into runtime state stores and architecture schematics. The primary runtime state stores include `SimulationState`, `ProducerState` and `ConsumerState`. The `SimulationState` tracks the system logs, present and past queue states, pending outputs and any simulation event like a failure or state surge. Conversely, the `ProducerState` and `ConsumerState` store their individual item type, the time it will be next available and feedback-related metrics like error integral, control signal and the latest feedback update time. The architecture schematics are stored in one centralised data class called `SimConfig`. It contains the simulation timeout, feedback direction, starting queue

occupancies and the processes. These processes are defined using either the `ProducerConfig` or the `ConsumerConfig`. These process configurations contain the processing times, input and output queues and feedback parameters like reference signal, proportional gain (k_p), integral gain (k_i) and transport delay.

The `sim_runtime.py` is an expansive file containing many essential functions. The most significant ones are the following:

- **run_simulation:** Is responsible for the progression of time in the simulation, always iterating time to the next earliest event. These events can be a process completion, the assignment of a process, an update to the feedback control or the addition of a new log. While most events occur asynchronously, updates to the feedback control and the log interval occur at constant times. This helps simulate the continuous nature of feedback and ensures that the diagrams follow a uniform format.
- **producer and consumer:** Process items corresponding to their allocations. When the processing of an item is complete, it updates its next ready time depending on the processing rate and whether feedback was applied. Lastly, a log of the process completion is made.
- **update_feedback_controllers:** Is used to compute the control signal (u) for the processes with feedback enabled. It determines the reference signal (r) for the process, then computes the feedback error for the signal following the equation in Section 3.3.2. Then lastly, it updates the control signal store with the latest version of PI control established in Section 3.3.3 by Equation 3.11. The input parameters dictate which type of feedback is applied, as it can either be PI control or PI control with transport delay. Additionally, the reference signal can either be applied to the input or output queue, depending on the selected feedback direction. If the process does not have a queue corresponding to the direction, the base processing rate is used.
- **apply_feedback_control_to_processing_time:** Provides the next available time for the process based on Equation 3.12 defined in Section 3.3.3. This form of application of feedback offers a linear mapping between the current processing rate and the latest control rate. Additionally, having bound the control signals (u) for the processes between -1 and 1 ensures that the processing rates remain in a realistic range of 0% to 200%.
- **plot_results:** Compiles both the throughput rate and the queue state graphs into one diagram based on the logs for the simulated processes and queue states.
- **run_stability_experiment:** Uses the stability configuration defined by the simulated experiment in the `sim_scenarios.py` to create three stability heat maps. These heat maps correspond to the cumulative queue standard deviation, step variability and drift. Then, for each heat map, a simulation is plotted below using the parameters that achieved the maximum values for that heat map. These three heat maps and their simulations are then combined into one diagram.

- **run_optuna:** Using the Optuna library, Bayesian optimisation is computed to effectively search for optimal parameters using an objective function. This function was chosen to maximise self-sustaining oscillations given the expected dynamics defined in Section 2.6. For each experiment, the architecture is already defined, and so the feedback parameters (k_p , k_i , and δ) define the search space. The function used to evaluate for oscillations parses the data, determines the oscillation height and penalises any changes in variation.

3.4.4 Simulation Time Progression Mechanism

Throughout the development of this simulation tool, the way in which time increments has gone through multiple versions. While there is a comprehensive description of the available simulation time mechanisms in Section 2.7.3, this was not the case when development was started. As a result, the initial simulation was real-time based using multiprocessing. While this worked, the dependence on real-time as the driver of simulation progression made comprehensive simulations take a long time. Additionally, multiprocessing introduces process scheduling, which can lead to small differences between multiple runs of the same setup.

To resolve this issues, the simulation was converted into simulated incremental time advances and then shortly after onto event-driven simulation time. Incrementing the simulation time solved the issues with the experiment length being correlated to real-time progression. However, it does still incur a similar issue to multiprocessing, where events are clumped into one time step. This then results in the order of process completions being inconsistent. This was thus fixed by introducing a pending outputs store for time progression, making the simulation event-driven. This output store could be used to increment to the exact times following a consistent order of process completions. This eliminated redundant progressions and event clumping. As a result, the final implementation of time progression is event-driven.

Chapter 4

Experiments

4.1 Experiment Objectives and Evaluation Metrics

4.1.1 Theoretical Dynamics Validation

The experiments in this study aim to empirically validate the expected dynamics that were established for each system’s architecture in the Approach chapter. To accomplish this, each experiment scenario is executed with the parameters that are expected to elicit the desired dynamics. If these expected dynamics manifest in the evaluation metrics, it demonstrates that the framework can accurately depict the established theoretical models. To obtain the parameters for foundational dynamics like bottlenecks and backpressure propagation, the values can be deterministically set. However, for more complex dynamics, tools like Optuna or the stability heat maps are used to optimise the parameters towards the desired dynamics.

4.1.2 Throughput and Queue Occupancy

There are two primary metrics used to evaluate each simulated system’s internal dynamics. The first measures the throughput of each process in the sequence, and the second reflects the queue occupancy state for each queue in the sequence. Both of these metrics capture the values at each defined time step, and as a result, when plotted onto a graph, these metrics can show the shifts in behaviour over time.

To compute the process throughput metric, the process completion time logs for each process are aggregated. Given the dynamics involved within the simulations, most process completions occur at different times, and thus an aggregation window is required to detect differences in the frequency of completions. If this window is too wide, microdynamics could be lost, and if the window is too small, dynamics could become noisy and highly volatile. As such, an aggregation window of 1 second was chosen because it aligns well with the base processing rates of 0.5 and 1 second.

The queue occupancy metric is a state variable in the domain of control theory, representing the momentum of the dynamics within the system. The metric is derived from the queue occupancy logs, which have a constant sampling rate of 1 second. Since this sampling rate

matches the aggregation window of the throughput, it is not only uniform, but also means that both data points correspond to the same exact simulation time.

The combination of both of these metrics is essential to fully express the simulation's state at each time step. While throughput provides precise information on the flow within the system, the queue occupancy shows the state of the system and the momentum of its internal stores. Having both of these metrics can, for instance, reveal if a stagnant throughput is a result of either starvation from the previous process or a bottleneck in the subsequent process.

4.1.3 Quantifying Stability

While throughput and queue occupancy can be used to analyse the dynamics for a single configuration, these metrics lack the ability to visualise many different configurations at once. This is where stability diagrams allow for the mapping of a system's stability within a range of values for each analysed parameter. Running this stability analysis using two parameters allows for the visualisation of how they interact on a 2D diagram. The parameters used for this mapping include the proportional gain (k_p), integral gain (k_i) and transport delay (δ). While other parameter combinations were explored, the primary driver of dynamics was the three parameters outlined.

To quantify stability, the queue occupancy was chosen as the metric in which each system's stability was defined. While throughput was considered, queue occupancy showed more consistent and longer-term trends. Additionally, the queue occupancy is a state variable, and so is its store of momentum. To ensure a comprehensive understanding of a system's stability, three stability metrics were introduced. These metrics are used to calculate a combined instability score based on the system's queue capacities. These metrics are as follows:

- **Standard deviation:** This metric detects how varied the data is from the reference signal. Additionally, variance is a driver for the Bullwhip Effect [13].
- **Step variability:** This measures the frequency of jitters or oscillations. This metric can identify systems that could have a low standard deviation from the reference signal, but could be highly jittery.
- **Drift:** This metric offers an insight into whether the observed dynamics contain an underlying drift. Even a small drift reaching the queue boundaries can result in unforeseen dynamics.

For each metric, the internal state of the system configuration that achieved the highest score is also depicted. This allows the heat map diagram defined in Section 3.4.3 to illustrate not only the stability map for multiple metrics, but also its most unstable versions. While the scores are architecture-specific, the combination of these stability metrics helps map internal system dynamics and shows their most unstable states.

4.2 Experiment Process

There are two approaches used to explore the dynamics within this study. One is by varying the sequence length, and the other is by running the system under different conditions. A combination of these two approaches was considered the most appropriate to discover the dynamics in an efficient manner. This means that for some experiments it is beneficial to explore a condition with multiple sequence lengths, but for others it might not be necessary. As a result, for the experiments where the sequence length provides no significant impact, using a standard sequence length allows for consistency and comparability. Given this, a sequence length of three was chosen for these experiments. This configuration contains one producer acting as the system source, one intermediate process and one consumer acting as the sink. This architecture is ideal as an imbalance or disturbance can be introduced into the middle process, and both the upstream and downstream dynamics can be observed. Having exactly one of each process type allows for clearer results without a loss in generalisability for such experiments not exploring sequence length. Given the resemblance of these composed systems to a factory line, the items within the experiments were themed as such. This gave rise to the flow of items being from Iron Ingot, to Iron Rods, to Iron Wire and more.

Additionally, for these unidirectional systems, the throughput in one process cannot exceed the previous process's throughput. As such, for some experiments, the first process is given a faster base processing rate than the subsequent ones. This configuration allows for the dynamics within the experiments not to be immediately limited by the source, as an initial increase in the queue occupancy might be required to induce more complex dynamics.

4.3 Open Loop Systems

4.3.1 Baseline Configuration

For the purposes of the open-loop experiments in this study, the following baseline parameters will be used:

Table 4.1: Summary of open-loop baseline parameters.

Parameter	Baseline Values		
Process Names	Process 1: Producer	Process 2: Producer and Consumer	Process 3: Consumer
Process Item Type	Iron Ingot	Iron Ingot $\rightarrow$ Iron Rod	Iron Rod
Base Processing Rates	2.0 items/s	2.0 items/s	2.0 items/s
Standard Simulation Time	60 seconds		
Queue Capacity	10 items (per queue)		
Initial Queue Occupancy	0 items (per queue)		

As illustrated by Table 4.1, the system contains three processes, each with a processing rate of 2.0 items per second. This uniform processing rate allows for a clean slate that the experiments can make minimal adjustments to. Each queue capacity is consistent to minimise different conditions influencing the dynamics. An experiment length of 60 seconds was found to be sufficient to showcase the dynamics.

4.3.2 Balanced Flow and Aggregate Throughput

The objective of this experiment is to verify that the processing rate of two parallel processes is equivalent to a process with a processing rate equal to the sum of the parallel processes. The expected behaviour is a uniform throughput, showing that in sequential systems, multiple processes simplify into one combined throughput. To implement this experiment, the open-loop baseline is modified by converting Process 2 into two parallel processes called Process 2.1 and Process 2.2, with each assigned a processing rate of 1.0 items/s. Additionally, the initial queue occupancy is set to 5 so that sink behaviours are not hidden.

4.3.3 Bottlenecks and Starvation

This experiment aims to explore the effects of a mismatch in processing rates. The expected result is the ability to induce both bottlenecks and starvation as discussed in Section 2.6.1. To achieve these dynamics, two separate simulations are used, both based on the open-loop baseline. For the bottleneck simulation, Process 2's processing rate is reduced to 1 item/s. While for the starvation simulation, the initial queue occupancy of the Iron Ingot queue is set to 10 and Process 1's processing rate is set to 1.0 item/s.

4.3.4 Backpressure Propagation

The final open-loop dynamics introduced in Section 2.6.1 is backpressure propagation. This dynamic necessitates the use of multiple queues, making it the first dynamic not achievable by an atomic system. The experiment is expected to have an internal rate mismatch that cascades from one process to another, either as saturations or depletions. Since the expectation is that these dynamics can propagate up the sequence, it would, in essence, break the rule of unidirectional architecture defined for atomic systems. To implement this experiment, an additional queue and its related process will be attached to the baseline sequence to provide a better visualisation of the propagation. The mismatch is a slower processing rate of 1.0 item/s and will be added to the new process at the end of the sequence. For this specific configuration, an upwards propagation of a bottleneck is expected.

4.4 Closed-Loop Systems

4.4.1 Baseline Configuration

For the purposes of the closed-loop experiments in this study, the following baseline parameters will be used:

Table 4.2: Summary of closed-loop baseline parameters.

Parameter	Baseline Values		
Process Names	Process 1: Producer	Process 2: Producer and Consumer	Process 3: Consumer
Process Item Type	Iron Ingot	Iron Ingot $\rightarrow$ Iron Rod	Iron Rod
Base Processing Rates	1.0 items/s	1.0 items/s	1.0 items/s
Standard Simulation Time	1000 seconds		
Queue Capacity	100 items (per queue)		
Initial Queue Occupancy	0 items (per queue)		
Feedback Direction	Output Queue		
Reference Signal (r)	50 items		
Feedback Parameter Bounds	Ranges and Resolution		
Proportional Gain (k_p)	0.0 to 1.0		
Integral Gain (k_i)	0.0 to 0.1		
Transport Delay (δ)	1.0 to 100.0 seconds		
Search Grid Resolution	25×25 (625 total simulations per map)		

Within Table 4.2, the baseline architecture of three processes with constant processing rates remains. However, the queue capacity and simulation times were increased for more granular results, making the dynamics easier to see and visualise. The output queue was chosen for the application of feedback. The alternative will be explored in Section 4.4.4. Additionally, for the use of PI-based feedback control, a reference signal of exactly half of the queue capacity was allocated. This promotes the state of the queues to be as far from starvation or blocking as possible.

For the stability heat maps, the table sets out the bounds and the resolution of the heat maps. To run the search space, each range being evaluated is equally divided by the search resolution, resulting in the comprehensive heat map. Using a search resolution of 25×25 was deemed to offer sufficient visual clarity of the underlying instability, while also running in less than a minute. The proportional gain (k_p) was chosen to have a range no higher than 1.0. This was because any higher values would induce a reaction larger than the error inducing it. The integral gain (k_i) accumulates the error over the span of the simulation, and so even a small gain can dominate the dynamics. Consequently, using empirical experimentations, a gain of 0.1 was found to be the limit of relative stability. Then the transport delay's maximum

range was set to 100 seconds. This represents 10% of the simulation duration and thus any longer delays would be unrealistic and would reduce the visibility of the dynamics.

4.4.2 Application of Baseline Values and Parameter Exploration

Unless stated otherwise, the experiments within this section will maintain the same underlying architecture as defined in Table 4.2. The feedback parameters (k_p , k_i , and δ) will be explored using two exploratory tools introduced in Section 3.4.2. These tools are the stability heat maps and the Optuna optimisation function. Using the heat maps, the most extreme configurations can be explored for each architecture, whereas the optimisation function finds the parameters that sustain oscillations the most.

4.4.3 Feedback Mechanisms

This experiment aims to explore the different effects feedback mechanisms have on the internal dynamics of a system. As described in Section 3.3.1, PI control has been developed for the purposes of this simulation framework with the option to apply transport delay to the feedback's error calculations. This experiment allows the exploration of higher-order and infinite-dimensional systems within control theory, introduced in Sections 2.3.2 and 2.3.3. Higher-order systems are achieved using PI control-based feedback, which results in ODEs. While infinite-dimensional systems apply transport delay to this feedback, resulting in DDEs.

To conduct this experiment, the following simulation configurations were explored:

- An atomic system with PI-controlled feedback to provide a baseline for the consequence of composition. This was implemented by removing the last process and its queue from the baseline configuration in Table 4.2.
- A sequential system with PI-controlled feedback to observe the effect of composition. To achieve this, the architecture was maintained from Table 4.2 with the transport delay disabled.
- A sequential system with PI-controlled feedback that contains transport delay. This allows for the comparison between a higher-order and an infinite-dimensional system. To implement this architecture, the baseline configuration from Table 4.2 was used with all of the feedback parameters enabled.

Each feedback parameter is derived and analysed using the tools discussed in Section 4.4.2. It is expected that all of the system configurations listed can achieve oscillations as outlined in Section 2.3.3. Additionally, it is also expected that an increase in sequence length and the addition of delay make the system less stable. Finally, as described in Section 2.6.3, DDEs are required for the formation of the Bullwhip Effect, and so the PI control with feedback delay is expected to be able to achieve this.

4.4.4 Input vs Output Feedback

This experiment will assess the difference caused by applying the reference signal to either the input queue or the output queue. Since these systems are interconnected, feedback will be applied to the selected queue in order for the feedback mechanism to drive its state towards the reference signal. The expectation is for the system to be able to reach stability and produce oscillations. To implement this experiment, the only alteration required to the baseline table (Table 4.2) is the change in feedback direction.

4.4.5 Sequence Length and Compositional Instability

This experiment will explore the effect of increasing the composition size of the system. Since each system introduces an additional feedback mechanism, the dynamics are expected to be more unstable, and as a result, fewer systems are expected to reach a steady state at the reference signal. Furthermore, as discussed in Section 2.6.3, an increased sequence length increases the chances for the creation of the Bullwhip effect. To implement this experiment, four state variables were used in total, meaning two more processes were added to the sequence. Transport delay is applied to this experiment as it is a prerequisite for the Bullwhip effect.

4.4.6 Damping Ratios and Self-Sustained Oscillations

This experiment follows Section 2.6.2's classification of closed-loop dynamics within control theory. It outlines that a closed-loop system can be determined using the damping ratio and can either be overdamped, underdamped or critically damped. As a result, the original objective of this experiment was to calculate this ratio within the framework and then show how the experiments were classified into the correct damping bands. However, given the deterministic network flow-based implementation, the correlation from the system architecture to the ratio was deemed unattainable for this study. Without a way to calculate this ratio, the damping dynamic was determined through empirical observations.

4.5 Exogenous Disturbances

4.5.1 Baseline Configuration

For the evaluation of exogenous disturbances, the simulation configurations are based on the architecture defined in previous experiments. To apply the disturbances to these systems, the following parameters will be used:

Table 4.3: Summary of baseline exogenous disturbance parameters.

Disturbances Parameters	Baseline Values
Failure Item Type	Iron Ingot
Failure Start	300.0 seconds
Failure End	350.0 seconds
Surge Item Type	Iron Ingot
Surge Trigger Time	400.0 seconds

While both failures and surges are types of disturbances, they will be explored separately to build an understanding of the effect they have on system stability. The following experiments explore how an atomic system compares to a sequential system in the event of a disturbance. As a result, for both failures and surges, the iron ingot was chosen to be the source of the disturbance. The failure halts the consumer process of the selected item, while the surge fully saturates the queue of the selected item. The times selected for the events were chosen to ensure both enough time for the simulation to enter into a self-sustained state, while leaving enough time for the resulting dynamics to play out.

4.5.2 Process Failure

To understand the effect that sequential composition has on a system's resilience to process failures, this experiment will compare the resultant dynamics between an atomic system and a composed system. These systems are based on the configuration definitions in Section 4.4.3. For the sequential system, the configuration without transport delay was chosen to align with the atomic configuration. Nonetheless, the atomic system is expected to be more resilient, while the composed system is expected to amplify the disturbance and may have a harder time stabilising. The failure applied to both systems is shown in Table 4.3.

4.5.3 Surge Impact

Expanding on the concept defined in the previous section, this experiment explores the effect of queue saturation on both an atomic and a sequential system. The atomic configuration to which the saturation is applied is the one defined in Section 4.4.3. For the sequential system, a new configuration was developed based on the configuration defined in Table 4.2 with the addition of one more process and queue. This new system allows the surge to be applied to the middle queue, providing a view of both the upstream and downstream dynamics. Additionally, this new composed system omits transport delay so that the systems are more equivalent. The expectation is that a surge provides a more manageable disturbance, and so both systems are anticipated to be able to return to a stable state relatively quickly.

Chapter 5

Results and Discussion

5.1 Experimental Correspondence

The sections within this chapter follow the same format as the Experiment Chapter. This means both the experiment and its results can be found under the same title.

5.2 Open Loop Systems

5.2.1 Balanced Flow and Aggregate Throughput

As shown in Figure 5.1, the throughput of both subsystems is instantly 2 items/s, and since the throughput is the same after settling, the queue occupancies maintain a steady state. This settling is due to the consuming processes removing one item per process at initialisation. This result confirms the expectation that for sequential systems, the process count is irrelevant and what matters is the aggregate throughput.

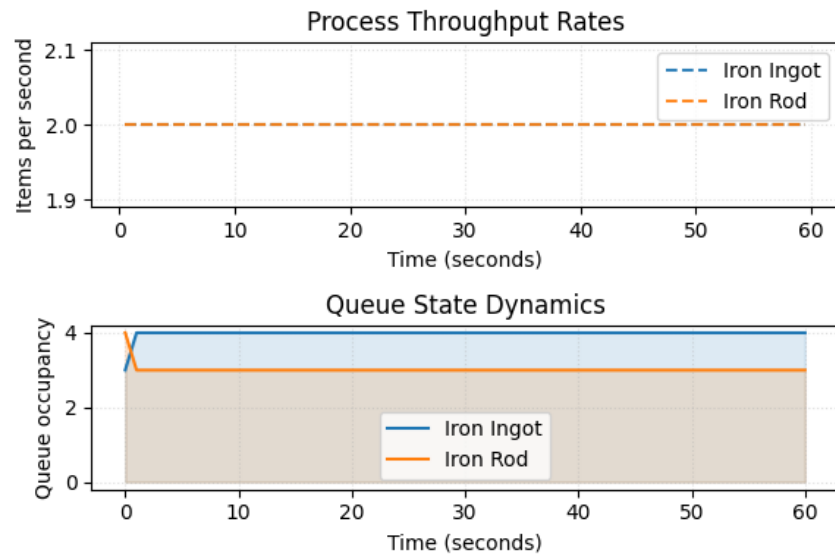


Figure 5.1: Open-loop system with parallel processes aggregates the throughput.

5.2.2 Bottlenecks and Starvation

In Figure 5.2, the presence of a bottleneck is clearly evident. The mismatch in the processing rate of Process 2 is visually apparent by the accumulation of Iron Ingots due to the insufficient consumption rate in comparison to the production of Iron Ingots. As a result, the system's processing rate settles at 1.0 items/s due to the constraint of Process 2.

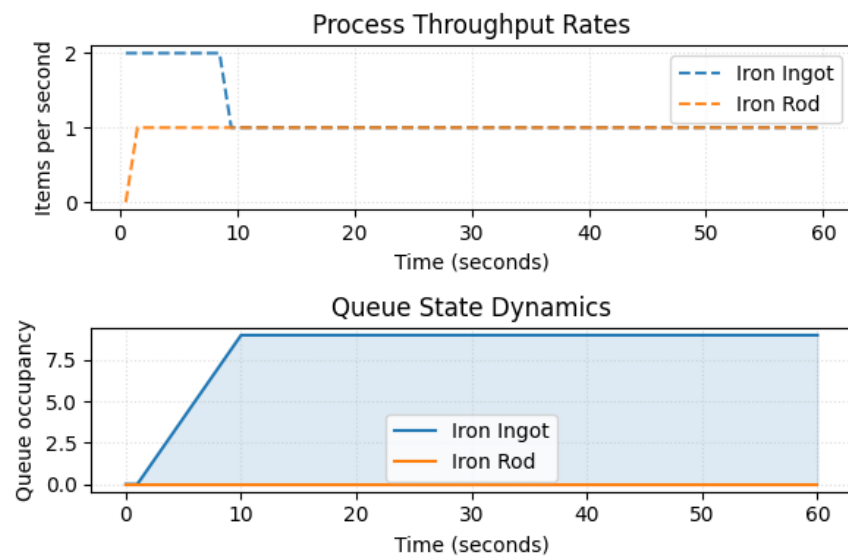


Figure 5.2: Open-loop system containing bottleneck dynamics.

Within Figure 5.3, the immediate queue occupancy depletion is visible, eventually resulting in system starvation. While the processing rate of Process 1 is maintained, the processing rate of the subsequent processes becomes partially underutilised.

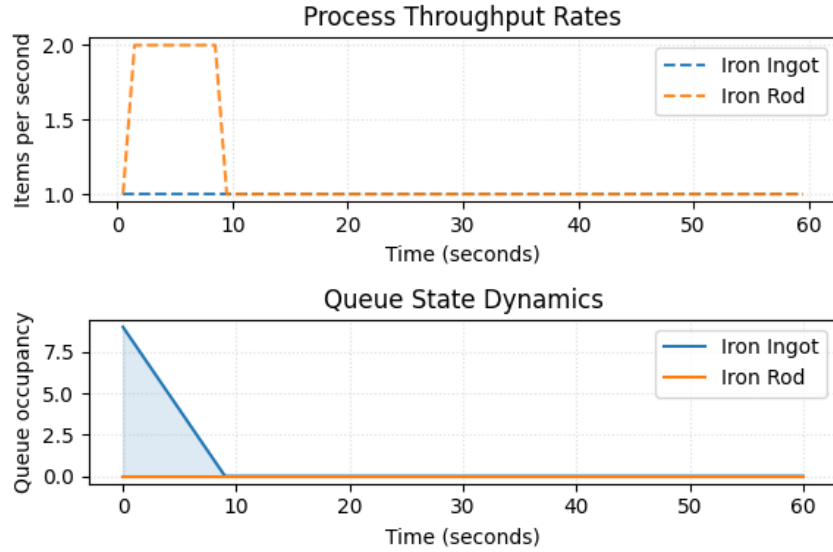


Figure 5.3: Open-loop system containing starvation dynamics.

Having the simulation result in the expected fundamental dynamics affirms the correctness of the simulation framework for open-loop architectures. Additionally, a theoretical expectation was introduced in Section 3.7, which states that an open-loop system's throughput converges to its minimum processing rate. This is evident in both experiments.

5.2.3 Backpressure Propagation

Visible in Figure 5.4 is the expected behaviour of a cascading bottleneck moving up the sequence discussed in Section 4.3.4. Given Iron Wire's slow consumption rate, its internal queue fully saturates over time, resulting in a bottleneck that limits the processing rate for Iron Rods. This slowdown in Iron Rods then progresses, eventually leading to the saturation of every queue and the convergence of the processing rate to the minimum.

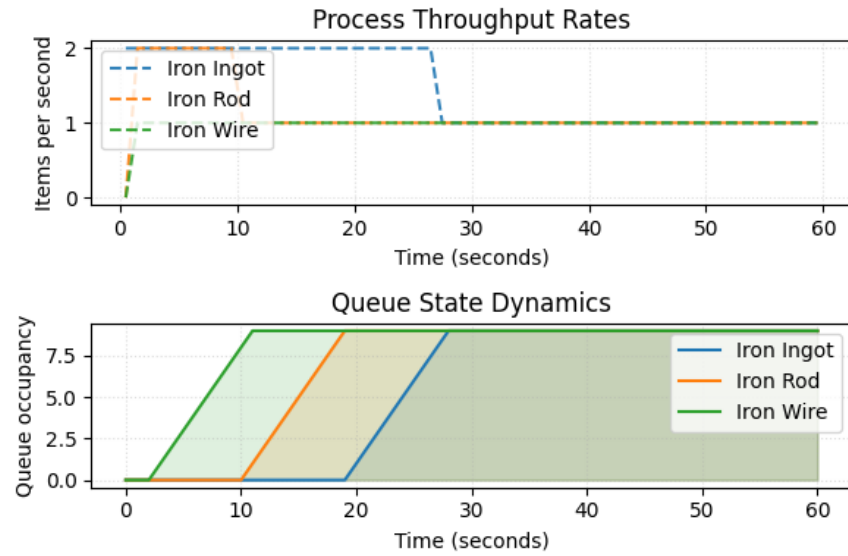


Figure 5.4: Open-loop system containing backpressure propagation dynamics.

5.3 Closed Loop Systems

The experiments in this section explore how system architecture and feedback configurations influence system dynamics and stability. To show these findings, two figures per experiment are used based on the tools defined in Section 4.4.2. This study primarily provides a simulation framework and a surface-level understanding of the apparent dynamics. As a result, the following sections do not dive into the precise details of the dynamics but instead focus on the mechanisms behind them.

5.3.1 Feedback Mechanisms

Atomic Closed-Loop PI Controlled System

To offer a baseline for the following composed architectures, an example of a closed-loop atomic system is shown in Figure 5.5. The specific parameters used were selected based on the oscillatory optimisation function from Section 3.4.3. The heat maps in Figure 5.6 show that given this architecture, approximately 40% of the search space is oscillatory.

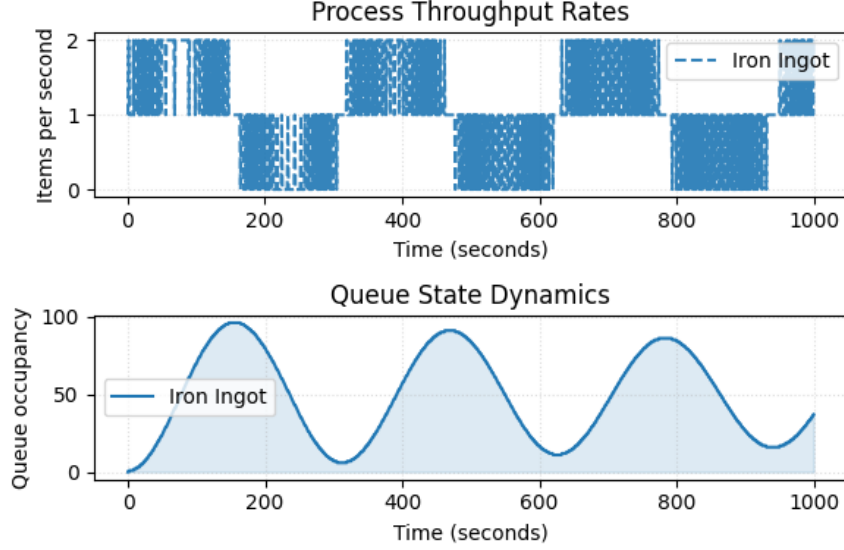


Figure 5.5: Closed-loop atomic system producing stable oscillations.

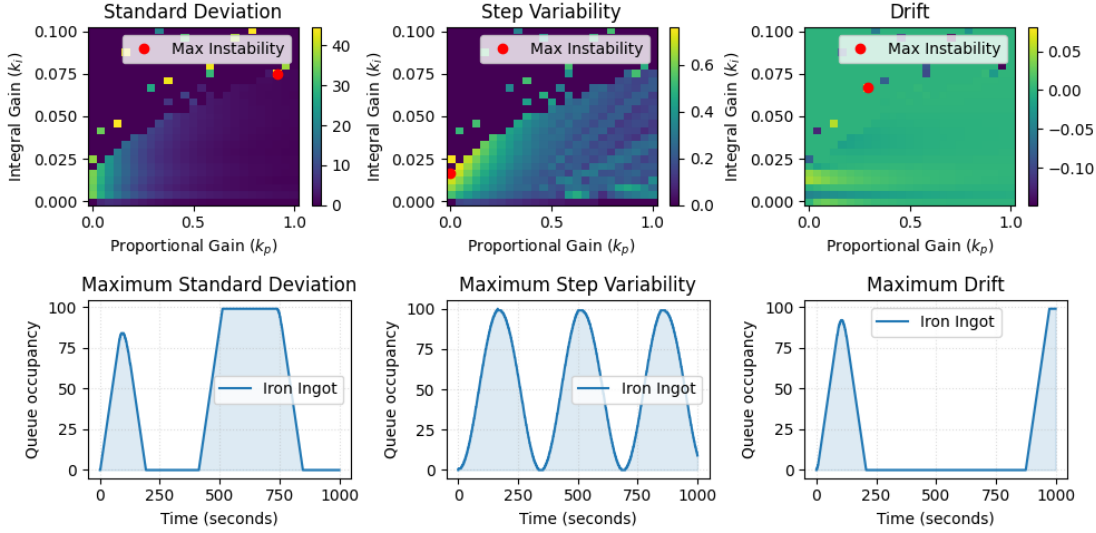


Figure 5.6: Closed-loop atomic system stability heat map.

Sequential Closed-Loop PI Controlled System

The expectations outlined for this configuration in Section 4.4.3 have been met by Figures 5.7 and 5.8. The expectations were that the addition of a new state variable and a second individual feedback mechanism would make the system more unstable than the atomic baseline. Interestingly, even though it is more erratic, the stability heat map still covers approximately the same region with a coverage of 40

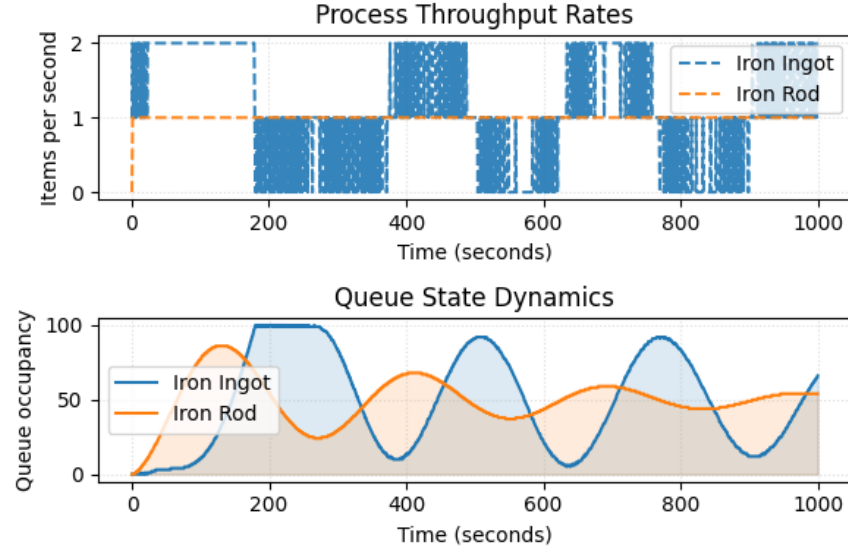


Figure 5.7: Closed-loop sequential system producing stable oscillations.

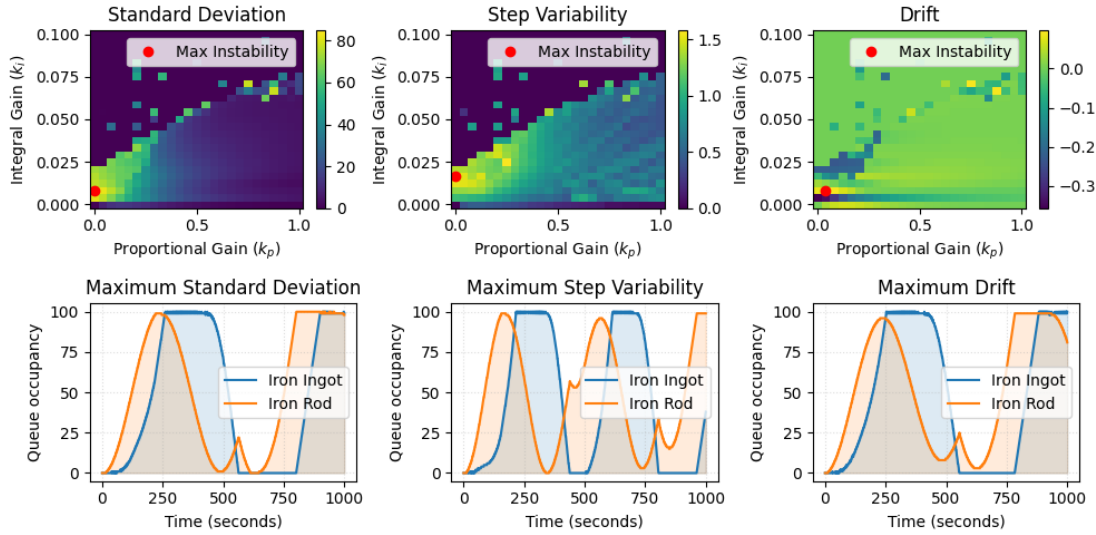


Figure 5.8: Closed-loop sequential system stability heat map.

Sequential Closed-Loop PI Controlled System with Transport Delay

The expectation for this system, stated in Section 4.4.3, was the potential appearance of the Bullwhip Effect, alongside more instability. However, this expectation was not fulfilled in the simulation Figures 5.9 and 5.10. While slightly more unstable, no significant change was apparent from the previous configuration, with no delay. The hypothesis is that a longer sequence length is necessary for the formation of the Bullwhip Effect.

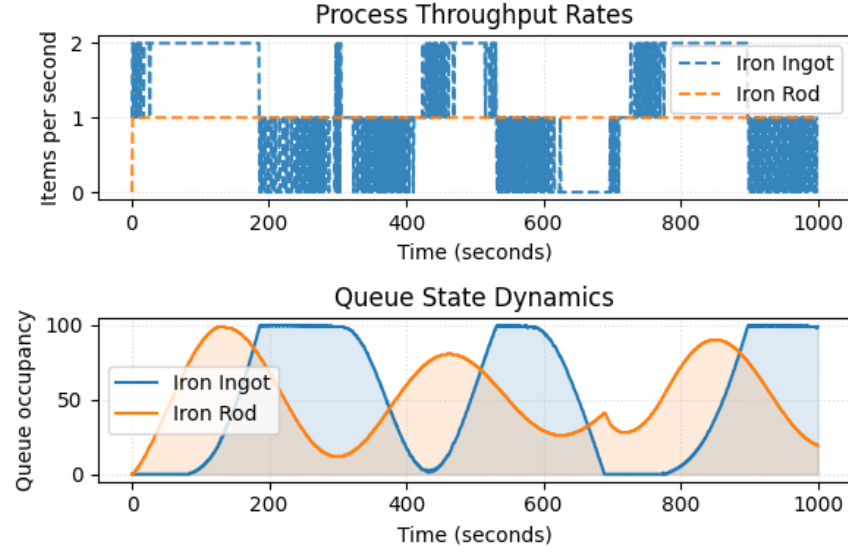


Figure 5.9: Closed-loop sequential system with transport delay producing stable oscillations.

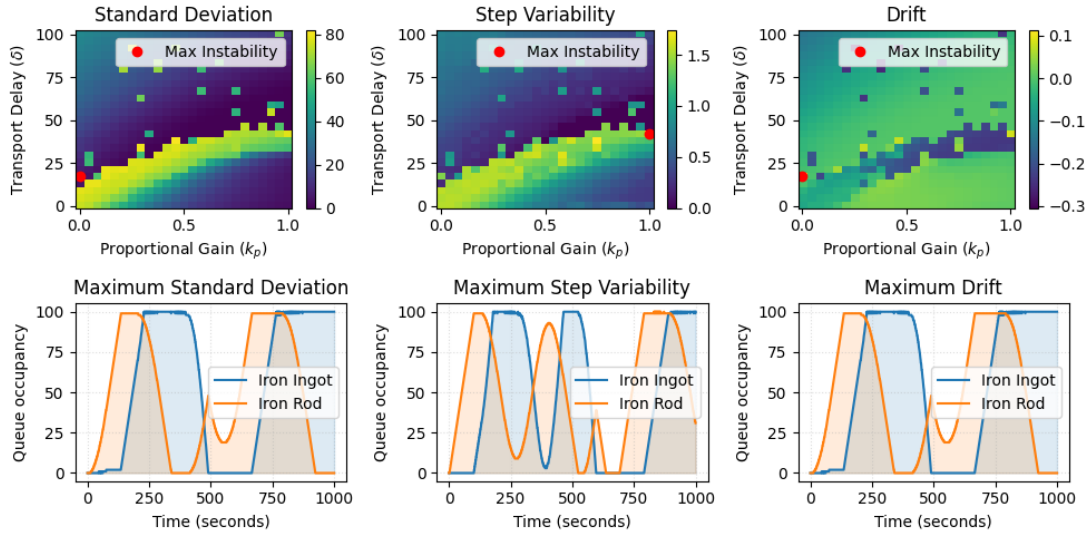


Figure 5.10: Stability heat map of Closed-loop sequential system with transport delay.

5.3.2 Input vs Output Feedback

While oscillations were expected for both directions in Section 4.4.4, the input configuration instead produced linear dynamics as shown in Figures 5.11 and 5.12. Since the output direction was set as the default, its results are located in Figures 5.7 and 5.8. Additionally, reflected by the standard deviation heat map, there is only a small amount of variance across which the input system reaches the reference point. This suggests that changes in feedback direction result in consequential changes to system behaviour.

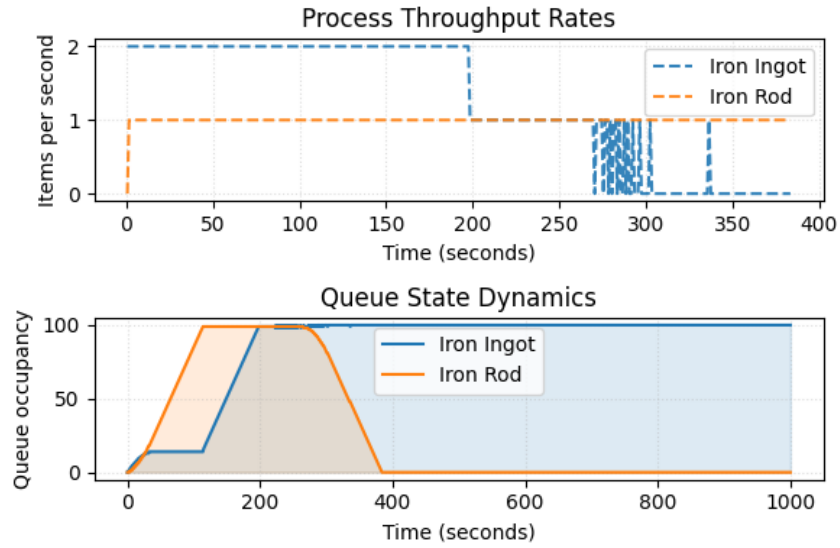


Figure 5.11: Closed-loop sequential system producing partially linear dynamics.

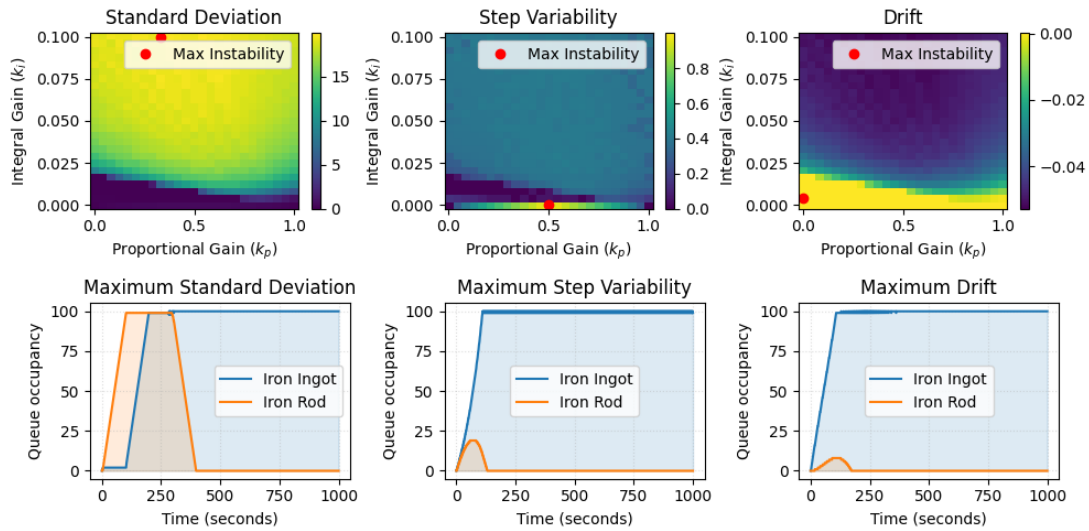


Figure 5.12: Stability heat map of closed-loop sequential system with input feedback.

5.3.3 Sequence Length and Compositional Instability

As anticipated in Section 4.4.5, increasing the sequence length from three processes in Section 5.3.1 to five processes, as shown in Figures 5.13 and 5.14, made the internal dynamics of the system more unstable. However, the desired Bullwhip Effect was still not obtained, even with this longer configuration. The suspected reason is that the effect might only manifest within very tight parameter ranges that were not found in this study. A potential future solution could be the use of a more specialised Optuna objective function for the search.

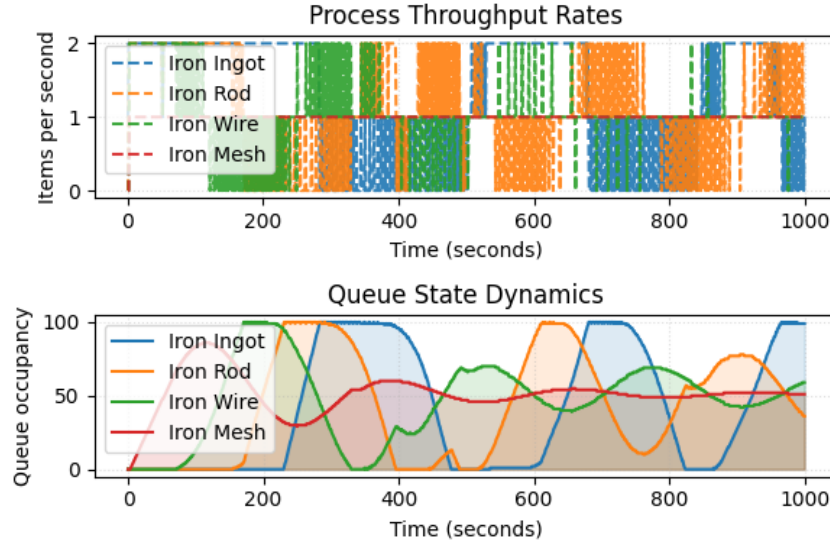


Figure 5.13: Closed-loop five-process system with transport delay producing chaotic dynamics

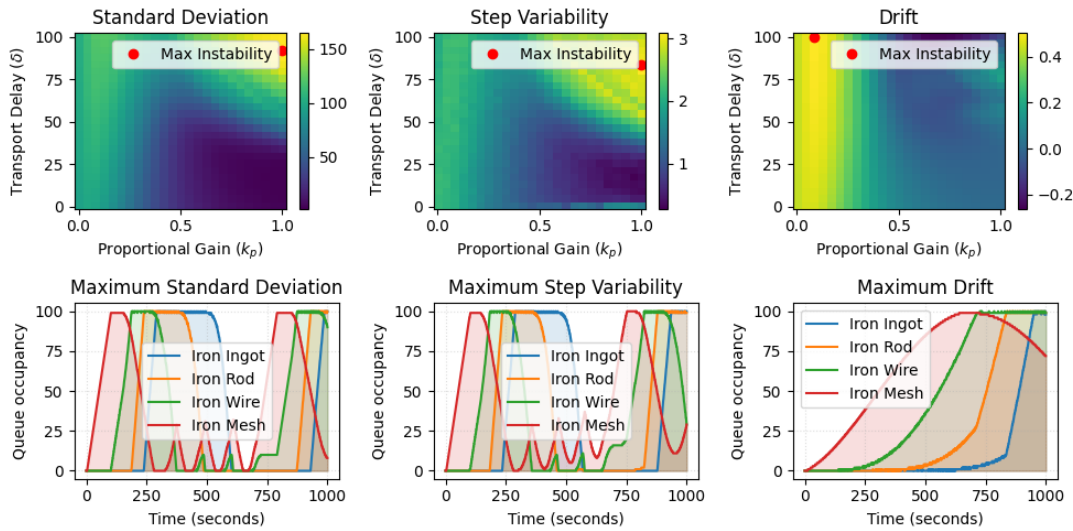


Figure 5.14: Stability heat map for a closed-loop five-process system with transport delay.

5.3.4 Damping Ratios and Self-Sustained Oscillations

Using empirical observations, the expected damping dynamic was achieved in Figure 5.15. To achieve this dynamic, an exceptionally low integral gain (k_i) of 0.0002 was used, and a relatively high proportional gain (k_p) of 0.5 was used. The suspected reason why the ratio between the two gain values is so large is that the damping dynamics accumulate the error integral in only one direction. This means that while the integral gain is still needed to eliminate steady-state errors, P control is the primary driving factor.

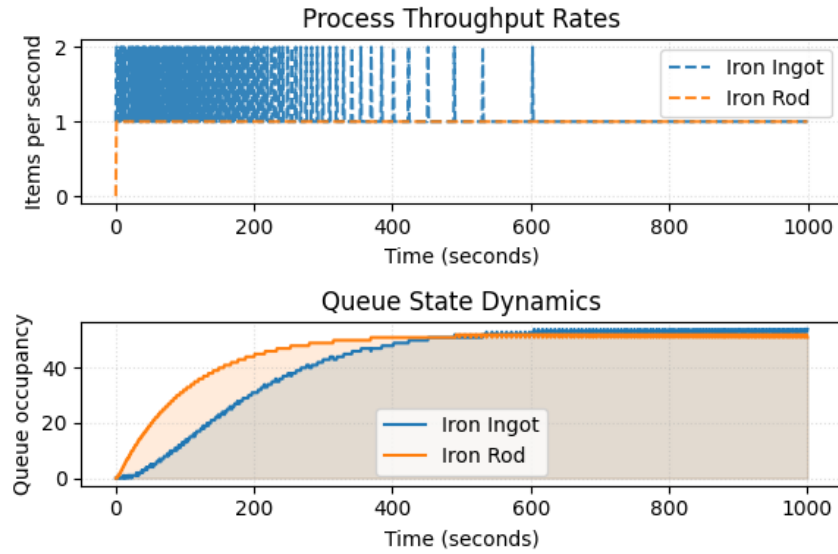


Figure 5.15: Closed-loop sequential system producing damping dynamics.

5.4 Exogenous Disturbances

5.4.1 Process Failure

Figures 5.16 and 5.17 show the process failure experiment defined in Section 4.5.2. While the expectation of returning to stability was met by the atomic system, this was not the case for the sequential system. Although the sequential system did try to recover, the hypothesised cause of the system-wide collapse is the excessive windup of the integral error.

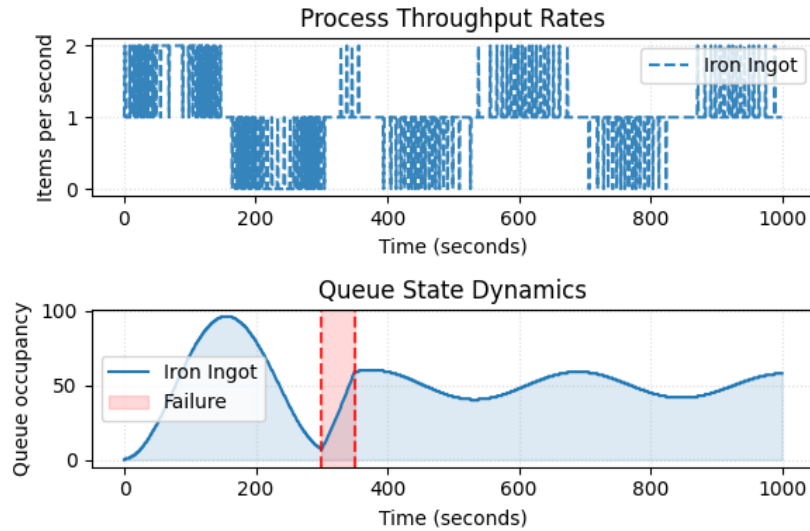


Figure 5.16: Closed-loop atomic system with process failure

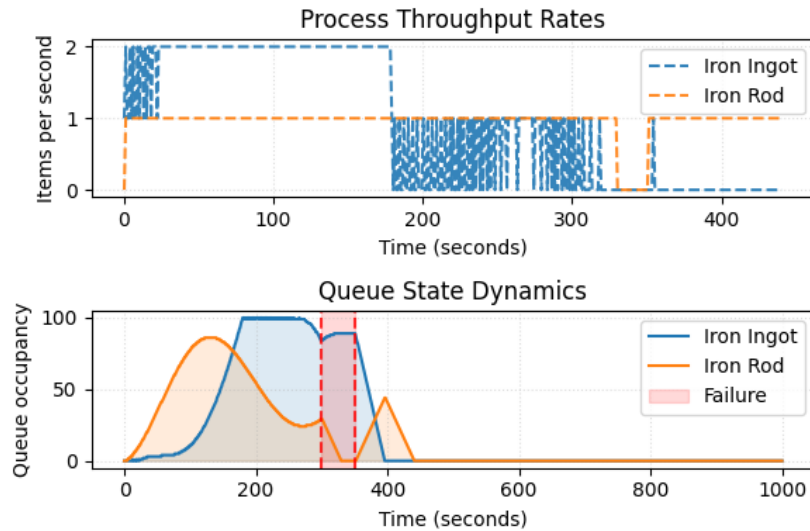


Figure 5.17: Closed-loop sequential system with process failure

5.4.2 Surge Impact

The impact a surge event has on the systems defined in Section 4.5.3 is shown in Figures 5.18 and 5.19. The expectation was for both systems to remain largely unaffected and to return to their previous dynamics. However, while the atomic system returned to its baseline dynamics, the sequential system attempted and failed to correct for the disturbance. This result is similar to the one observed in the failure experiment (Section 5.4.1).

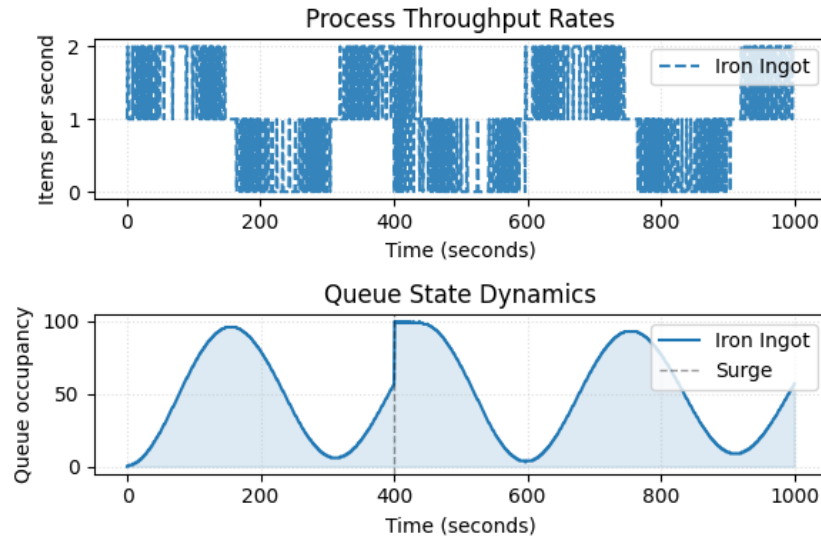


Figure 5.18: Closed-loop atomic system with a queue occupancy surge

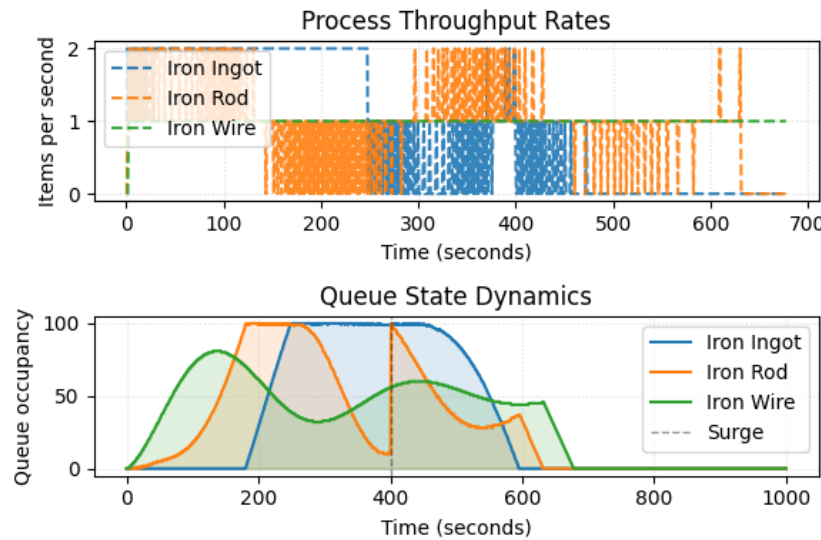


Figure 5.19: Closed-loop sequential system with a queue occupancy surge

5.5 Results and Discussion of Limitations

As demonstrated by the many successful experiments in this chapter, the simulation framework is able to simulate a multitude of different producer-consumer architectural configurations under many different scenarios. While this implementation provides a comprehensive way to simulate deterministic, item-based network flows for both ODEs and DDEs, there are some potential improvements.

The two primary framework improvements are the implementation of an anti-windup function and the creation of an objective function to isolate the Bullwhip Effect. The disturbances experiments in Section 5.4 demonstrated the increased instability caused by composition and multiple feedback loops. However, integral windup was most likely the cause for the system's never recovering after the disturbance. This means having anti-windup in the feedback mechanism could show a more realistic recovery for the system. The culmination of components like sequential composition, decentralised feedback and transport delay is expected to allow the manifestation of the Bullwhip Effect. While the effect was not found in this study, it does not mean that it is impossible within the current framework. Consequently, a more specialised Optuna objective function (Section 4.4.2) would have a higher chance of finding the parameters causing it. Alternatively, established literature can be used to provide the configurations that are able to induce the effect. However, given that this framework relies on discrete item-flows, the literature-derived values may require adjustment, which could be done via the Optuna objective function. Additionally, exploring real-world parameters would further showcase the benefits of the predictive capabilities of the framework.

Chapter 6

Conclusions and Future Work

6.1 Conclusion

This dissertation has successfully developed a simulation framework capable of simulating sequentially composed producer-consumer systems, visualising their internal dynamics, and classifying those dynamics. The possible application of this framework in the factory-line environment offers a grounded use case to understand the limitations of sequential systems. The system architectures this dissertation explores have been established in the literature from both control theory and queueing theory. The correlations offered by these domains provided a way to not only define a system's internal components, but also the expected dynamics each system is predicted to exhibit. This allows for the experiments to be based on the derived expectations and the results to be an empirical comparison between expectations and the resultant dynamics. Additionally, since the system architectures in this study have been expressed as both ordinary and delay differential equations (ODEs and DDEs), extensions of this work are mathematically straightforward. While no new dynamics were found outside of the established expected dynamics, the framework was able to reproduce almost every expected dynamic. The only exception was the Bullwhip Effect [13], the absence of which was further analysed and discussed in Section 5.5.

6.2 Future Work

The next two avenues of future work have been established as improving on the limitations discussed in Section 5.5 and the introduction of the Primal-Dual algorithm detailed in Section 2.5. The two improvements identified in the limitations section are the application of an anti-windup mechanism to the feedback for enhanced realism and using literature and Optuna to find parameters inducing the Bullwhip Effect (Section 2.6.3). Additionally, the Primal-Dual algorithm is a potential replacement for the PI control used for the process feedback. This economics algorithm applies feedback control, not based on the reference signal (r) as PI control does, but instead based on price and utility maximisation. Given that real-world systems are very often constrained by finances and market dynamics, the addition of Primal-

Dual-based feedback would allow the framework to model more complex supply chain dynamics.

Bibliography

- [1] K. Jeffay, “The real-time producer/consumer paradigm: A paradigm for the construction of efficient, predictable real-time systems,” in *Proceedings of the 1993 ACM/SIGAPP Symposium on Applied Computing: States of the Art and Practice*, SAC ’93, (New York, NY, USA), pp. 796–804, Association for Computing Machinery, Mar. 1993.
- [2] E. W. Dijkstra, “Cooperating Sequential Processes,” *The Origin of Concurrent Programming*, 1968.
- [3] A. B. Downey, *The Little Book of Semaphores*. Green Tea Press, 2016.
- [4] F. Szidarovszky and C. Chiarella, “The asymptotic behavior of dynamic producer-consumer systems,” *Mathematical and Computer Modelling*, vol. 39, pp. 1297–1312, June 2004.
- [5] A. Bruns, “Beyond the producer/consumer divide: Key principles of produsage and opportunities for innovation,” *The New Development Paradigm: Education, Knowledge Economy and Digital Futures*. New York: Peter Lang, 2014.
- [6] S. Chopra, *Supply Chain Management: Strategy, Planning, and Operation, Global Edition*. Harlow, UNITED KINGDOM: Pearson Education, Limited, 2019.
- [7] L. Kleinrock, *Queueing Systems Volume 1 Theory*. John Wiley & Sons, 1975.
- [8] K. J. Åström and R. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [9] J. D. Sterman, “Systems thinking and modeling for a complex world,” *Management*, vol. 6, no. 1, pp. 7–17, 2000.
- [10] H. L. Smith, *An Introduction to Delay Differential Equations with Applications to the Life Sciences*, vol. 57. springer New York, 2011.
- [11] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, “Rate control for communication networks: Shadow prices, proportional fairness and stability,” *Journal of the Operational Research Society*, vol. 49, pp. 237–252, Mar. 1998.

- [12] S. Low and D. Lapsley, “Optimization flow control. I. Basic algorithm and convergence,” *IEEE/ACM Transactions on Networking*, vol. 7, pp. 861–874, Dec. 1999.
- [13] H. L. Lee, V. Padmanabhan, and S. Whang, “Information Distortion in a Supply Chain: The Bullwhip Effect,” *Management Science*, vol. 43, pp. 546–558, Apr. 1997.
- [14] C. Segovia, “Petri nets in epidemiology,” *Journal of Mathematical Biology*, vol. 91, p. 40, Oct. 2025.
- [15] S. Nanz and C. A. Furia, “A Comparative Study of Programming Languages in Rosetta Code,” in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, pp. 778–788, May 2015.
- [16] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, (Anchorage AK USA), pp. 2623–2631, ACM, July 2019.
- [17] “PEP 557 – Data Classes | peps.python.org.” <https://peps.python.org/pep-0557/>.
- [18] J. Brownlee, “Thread Producer-Consumer Pattern in Python,” Apr. 2022.
- [19] J. Brownlee, “Multiprocessing Queue in Python,” May 2022.
- [20] J. Banks, *Discrete Event System Simulation*. Pearson Education India, 2005.